

大连理工大学本科毕业论文（设计）

基于大语言模型微调的视频码率自适应优化

Adaptive Video Bitrate Optimization via Fine-Tuning Large Language Models

学 院： 计算机科学与技术学院

专 业： 计算机科学与技术

学 生 姓 名： 王祥豪

学 号： 20221073015

指 导 教 师： 袁功霖

评 阅 教 师： 马野

完 成 日 期： 2026 年 6 月 1 日

大连理工大学

Dalian University of Technology

原创性声明

本人郑重声明：本人所呈交的毕业论文（设计），是在指导老师的指导下独立进行研究所取得的成果。毕业论文（设计）中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究成果做出重要贡献的个人和集体，均已在文中以明确方式标明。

本声明的法律责任由本人承担。

作者签名：

日 期：

关于使用授权的声明

本人在指导老师指导下所完成的毕业论文（设计）及相关的资料（包括图纸、试验记录、原始数据、实物照片、图片、录音带、设计手稿等），知识产权归属大连理工大学。本人完全了解大连理工大学有关保存、使用毕业论文（设计）的规定，本人授权大连理工大学可以将本毕业论文（设计）的全部或部分内容编入有关数据库进行检索，可以采用任何复制手段保存和汇编本毕业论文（设计）。如果发表相关成果，一定征得指导教师同意，且第一署名单位为大连理工大学。本人离校后使用毕业论文（设计）或与该论文直接相关的学术论文或成果时，第一署名单位仍然为大连理工大学。

论文作者签名：

日 期：

指导老师签名：

日 期：

摘要

随着在线视频和实时流媒体业务规模不断扩大，动态网络环境下的视频传输需要根据带宽、缓冲区状态和视频分片特征实时选择合适码率，以提升用户体验质量（Quality of Experience, QoE）。传统自适应码率（Adaptive Bitrate, ABR）算法依赖固定规则、短期带宽预测或专用强化学习模型，在网络条件变化较大时存在泛化能力不足、模型设计成本较高等问题。针对上述问题，本文基于 NetLLM 与自适应码率视频传输仿真框架，设计并实现了一种基于大语言模型微调与任务适配的码率决策方法。

本文围绕 ABR 数值状态与大语言模型语义空间不匹配、历史反馈难以稳定利用、短期带宽趋势建模不足三个问题，分别设计语义重编程状态编码、结合历史动作与历史奖励的上下文预对齐机制，以及轻量化历史多尺度建模模块。实验以公开视频网络轨迹和固定视频配置为测试对象，并补充同源验证子集和跨视频配置实验；评价指标包括平均奖励、平均码率、重缓冲时间和平滑性惩罚。结果表明，在主测试配置下，本文最终模型在 5 个随机种子下取得 0.8967 ± 0.0475 的平均奖励，高于基线算法的 0.8587 ± 0.0243 ，体现出平均 QoE 提升。进一步分析表明，该提升并非在所有轨迹上稳定占优，且跨视频配置下模型对分片大小分布变化仍较敏感，说明当前方法在稳定性和泛化能力方面仍有不足。

关键词：自适应码率；大语言模型；语义重编程；体验质量；视频传输

Adaptive Video Bitrate Optimization via Fine-Tuning Large Language Models

Abstract

With the continuous expansion of online video and real-time streaming services, video transmission in dynamic network environments needs to select appropriate bitrates in real time according to bandwidth, buffer status, and video chunk characteristics, to improve Quality of Experience (QoE). Traditional adaptive bitrate (ABR) algorithms usually rely on fixed rules, short-term bandwidth prediction, or dedicated reinforcement learning models. When network conditions change significantly, these methods may suffer from limited generalization ability and high model design costs. To address these problems, this thesis designs and implements a bitrate decision-making method based on large language model fine-tuning and task adaptation, using NetLLM and an adaptive bitrate video transmission simulation framework.

This thesis focuses on three problems: the mismatch between ABR numerical states and the semantic space of large language models, the unstable use of historical feedback, and insufficient modeling of short-term bandwidth trends. To solve these problems, semantic reprogramming state encoding, a context pre-alignment mechanism combining historical actions and historical rewards, and a lightweight historical multi-scale modeling module are designed. Experiments are mainly conducted on public network traces and fixed video configurations, with additional tests on a validation subset from the same data source and cross-video configurations. The evaluation metrics include average reward, average bitrate, rebuffering time, and smoothness penalty. The results show that, under the main test configuration, the final model achieves an average reward of 0.8967 ± 0.0475 over five random seeds, which is higher than 0.8587 ± 0.0243 of the baseline algorithm, indicating an improvement in average QoE. Further analysis shows that the improvement is not consistently achieved on all traces, and the model remains sensitive to changes in the video chunk size distribution under cross-video configurations. This indicates that the current method still has limitations in stability and generalization.

Key Words : Adaptive bitrate; Large language model; Semantic reprogramming; Quality of Experience; Video transmission

目录

摘要	I
Abstract	II
引言	1
1 相关技术与理论基础	5
1.1 自适应码率视频传输基本原理	5
1.2 ABR 决策问题建模	5
1.3 强化学习与离线策略学习	6
1.4 Transformer 与大语言模型	6
1.5 参数高效微调与 LoRA	7
2 需求分析与总体设计	8
2.1 系统目标与设计原则	8
2.2 系统总体架构	8
2.3 实验流程设计	9
2.4 评价指标设计	9
3 基于大语言模型适配的 ABR 决策方法设计	11
3.1 基础策略模型设计	11
3.2 语义重编程状态编码	12
3.3 上下文预对齐机制	13
3.4 轻量化历史多尺度建模	14
3.5 训练目标与优化方法	17
4 系统实现	18
4.1 实验环境与工程结构	18
4.2 ABR 环境与基线测试实现	18
4.3 经验池与数据集构造	19
4.4 状态编码与策略模型实现	20
4.5 训练、验证与测试脚本实现	21
5 实验设计与结果分析	22
5.1 实验设置	22
5.2 对比方法与评价指标	22
5.3 主实验结果分析	23
5.4 QoE 组成指标分析	25

5.5	消融实验与稳定性分析	25
5.6	分布级与逐轨迹分析	28
5.7	同源网络轨迹验证子集补充实验	29
5.8	跨视频配置补充实验	30
5.9	实验局限与方法论反思	33
结 论		35
参 考 文 献		36
修改记录		38
致 谢		39

引言

随着移动互联网、在线视频平台、短视频业务、云游戏和实时流媒体服务的快速发展，视频数据已成为网络传输中的重要业务类型。与普通文件下载不同，视频流媒体传输不仅要求数据能够完整到达，还要求在播放过程中保持较高的视频清晰度、较低的卡顿时间和较稳定的画质变化。在实际网络环境中，用户终端所处的带宽条件、链路时延、网络拥塞程度和设备缓存状态往往持续变化。如果码率选择过高，客户端可能因下载速度不足而产生重缓冲，破坏观看连续性；如果码率选择过低，则会降低画面质量，影响用户对视频内容的主观感受。因此，如何在动态网络环境下根据网络状态、缓冲区状态和视频分片特征实时选择合适码率，是自适应码率视频传输领域的重要研究问题。

自适应码率（Adaptive Bitrate, ABR）技术的核心目标是在有限且波动的网络资源条件下优化用户体验质量（Quality of Experience, QoE）。在典型 HTTP 自适应流媒体系统中，视频通常被切分为多个连续分片，每个分片预先编码为多个不同码率版本。客户端在下载下一个视频分片前，需要根据当前缓冲区长度、历史吞吐量、上一分片下载时间、可选码率集合以及未来视频分片大小等信息进行决策。该问题本质上是一个具有时序依赖和多目标权衡特征的序列决策问题：一方面，系统希望选择较高码率以提升视频质量；另一方面，又需要控制重缓冲时间和码率频繁切换带来的体验损失。已有研究表明，视频质量、重缓冲时间和码率切换等因素共同影响 HTTP 自适应流媒体系统中的用户体验质量^[1]。由于网络带宽具有随机性和突发性，单纯依赖当前状态或短期估计往往难以获得稳定的长期效果，这使得 ABR 决策具有较强的研究价值和工程意义。

围绕 ABR 问题，国内外研究已经形成了较为丰富的方法体系。从国外研究来看，视频自适应码率优化方向发展较早，整体上经历了从传统规则方法、控制优化方法到学习型方法的发展过程。传统 ABR 方法主要包括基于缓冲区、基于带宽预测和基于控制理论的方法。其中，BBA 根据客户端缓冲区占用情况直接选择码率，具有结构简单、实现方便和稳定性较好的特点^[2]；BOLA 从效用优化角度出发，将视频质量提升与卡顿风险控制统一到优化目标中，在码率收益和缓冲区安全之间建立权衡关系^[3]；Robust MPC 则利用历史吞吐量预测未来带宽，并通过模型预测控制在有限时间窗口内选择综合收益较优的码率序列，从而提高决策的前瞻性^[4]。随着机器学习方法的发展，Pensieve 将 ABR 建模为强化学习问题，通过神经网络从网络状态中直接学习码率决策策略^[5]；Comyco 引入模仿学习思想提高训练效率^[6]；Genet 利用自动课程生成机制增强环境适应性^[7]。此外，Kairos 沿着“预测增强+决策优化”的方向推进，在 MPC 框架中引入面向流媒体业务特性的吞吐预测与不确定性控制^[8]；也有研究从自学习客户端设计和边缘计

算辅助决策角度拓展学习型 ABR 的应用范围^{[9][10]}。总体来看,传统方法具有可解释性强、部署成本低等优点,但对复杂网络波动和跨场景变化的适应能力有限;学习型方法能够减少人工规则设计依赖,但通常需要大量经验数据,且性能容易受训练环境分布、奖励函数设置和模型结构选择影响。

近年来,大语言模型开始从自然语言处理扩展到时间序列分析、网络管理和决策优化等任务。NetLLM 提出将大语言模型适配到网络任务中,使大模型能够处理网络状态、协议行为和控制决策等任务,并在自适应码率等场景中展现出应用潜力^[11]。TIME-LLM 等研究表明,通过重编程方式可以将连续时间序列信号映射到大语言模型可处理的表示空间,从而发挥大模型在序列模式建模和跨任务迁移方面的能力^[12]。除大语言模型适配外,时间序列多尺度建模也为本文提供了重要启发。TimeMixer 从多尺度混合角度分析时间序列变化,指出真实时间序列在不同采样尺度下呈现不同模式,并通过可分解的多尺度混合结构整合不同尺度下的时间信息^[13]。这些研究说明,大语言模型并不局限于自然语言生成任务,也可能通过适当的输入编码、语义映射和参数高效微调方式迁移到具有明显时序特征的网络决策任务中;同时,ABR 中的历史吞吐量、缓冲区变化和下载时延序列也不应只从单一时间尺度理解,而应考虑局部波动与短期趋势之间的共同作用。

从国内研究来看,视频自适应码率优化方向起步相对较晚,但随着智能视频传输、网络优化和人工智能技术的发展,相关研究正在逐步增加。现有国内研究主要集中在 QoE 优化、强化学习码率控制以及动态网络环境下的自适应策略设计等方面,更加关注算法在真实网络环境中的可用性与稳定性。杜丽娜等对基于强化学习的移动视频流业务码率自适应算法进行了系统总结,梳理了强化学习在状态建模、奖励设计和策略优化方面的应用进展^[14];肖强等分析了强化学习在自适应视频码率控制算法中的应用,说明强化学习方法在动态网络条件下进行码率选择具有一定可行性^[15];宋靳铎等从 HTTP 自适应流媒体直播系统出发,研究了用户体验质量优化问题,强调吞吐量、缓冲区、直播时延和码率切换等因素对播放体验的影响^[16]。此外,熊丽荣等研究了基于 Q-learning 的 HTTP 自适应流码率控制方法^[17],夏勃等提出了动态自适应 HTTP 流媒体码率切换算法^[18],还有研究面向短视频流媒体场景引入多智能体强化学习和专家指导机制^[19]。这些研究说明,国内在视频传输 QoE 优化和学习型码率控制方面已经形成一定基础,也为本文选取平均码率、重缓冲时间、码率平滑性和综合 QoE 等评价指标提供了参考。

综合已有研究可以看出,将大语言模型引入 ABR 决策仍面临若干关键问题。首先,ABR 状态主要由码率、缓冲区、吞吐量、下载时延、视频分片大小和剩余分片数量等数值特征组成,与大语言模型原本擅长处理的自然语言输入存在明显差异,如何构造适

合大模型处理的状态表示是首要问题。其次，ABR 决策不仅依赖当前网络状态，还与历史动作、历史奖励和短期带宽变化趋势密切相关；其中，历史吞吐量序列既包含相邻分片之间的快速波动，也可能包含一段时间内的趋势性变化，单一尺度的历史表示难以同时刻画这两类信息。再次，大语言模型参数规模较大，而 ABR 场景对训练成本、推理效率和系统可复现性具有较高要求，因此需要采用参数高效微调和轻量化结构设计，避免直接全量微调带来的计算开销。最后，现有大模型网络任务研究仍处于发展阶段，针对 ABR 这类实时性强、多源状态输入明显、QoE 目标复杂的任务，仍需要结合具体场景进行方法设计和实验验证。

基于上述背景，本文围绕“基于大语言模型微调的视频码率自适应优化”展开研究。本文的研究范围限定在自适应码率视频传输仿真环境中的客户端码率决策问题，重点关注如何将大语言模型适配到 ABR 任务，使其能够根据网络状态、缓冲区状态和视频分片特征输出合理的码率选择。本文并非从零构建一个全新的通用视频传输系统，而是在已有 ABR 仿真框架和 NetLLM 适配思想基础上，面向 ABR 任务特点进行模型设计、模块改进和实验分析。具体而言，本文首先搭建统一的 ABR 实验流程，完成网络轨迹加载、经验池生成、离线训练、在线验证和测试评价等环节；然后对传统规则方法、模型预测控制方法和强化学习方法进行整理与对比，为大模型方法的评价提供基线；在模型设计方面，本文采用语义重编程思想，将 ABR 数值状态映射到大语言模型表征空间，并结合历史动作、历史奖励和历史吞吐量变化信息增强模型对当前决策上下文和短期网络变化趋势的建模能力。其中，历史吞吐量建模部分借鉴 TimeMixer 等时间序列方法中的多尺度分析思想，但不直接移植其预测框架，而是结合 ABR 在线决策的低开销需求设计轻量化历史多尺度模块，用于辅助模型感知局部带宽波动与短期趋势。最后，通过对比实验和消融实验分析所设计方法在综合 QoE 指标下的有效性。

本文工作的核心主线是：针对 ABR 数值状态与大语言模型语义空间不匹配、历史反馈难以稳定利用、短期带宽趋势建模不足三个问题，分别设计语义重编程、上下文预对齐和轻量化多尺度历史建模模块。具体而言，第一，本文将 ABR 决策限定为客户端侧分片级码率选择问题，不涉及服务器调度、编码器优化和真实播放器系统实现，从而保证研究对象边界清晰，并使实验指标的计算方式保持一致。第二，本文不是简单将数值状态直接输入大语言模型，而是围绕 ABR 状态的数值性、时序性和决策反馈性设计三类任务适配机制，使模型能够区分缓冲区压力、吞吐趋势、分片下载成本和历史奖励反馈。第三，本文在主测试配置之外，补充多随机种子、逐轨迹分布、同源验证子集和跨视频配置实验，用更严格的统计分析说明方法的平均性能收益、适用条件和局限性。

本文研究具有一定的理论探索意义和工程应用价值。从理论角度看，本文尝试将大语言模型的表示能力与 ABR 序列决策问题相结合，探索数值型网络状态到大模型表示空间的映射方式；从工程角度看，本文关注模型训练流程、基线复现、特征编码和在线测试等完整实验链路，强调方法的可实现性和可复现性。

本文后续内容安排如下：第 1 章介绍相关技术基础；第 2 章分析系统需求并给出总体设计；第 3 章阐述基于大语言模型适配的 ABR 决策方法；第 4 章说明系统实现过程；第 5 章给出实验设计与结果分析；最后对全文工作进行总结，并讨论当前方法存在的不足及未来改进方向。

1 相关技术与理论基础

1.1 自适应码率视频传输基本原理

随着在线视频业务的发展，传统固定码率视频传输方式逐渐难以适应复杂多变的网络环境。在早期流媒体系统中，视频通常以单一码率进行传输，当网络带宽下降时容易出现播放卡顿，而带宽充足时又无法充分利用网络资源。为解决这一问题，HTTP 自适应流媒体技术逐渐成为主流方案。

目前应用较广泛的 HTTP 自适应流媒体协议主要包括 HLS（HTTP Live Streaming）和 MPEG-DASH（Dynamic Adaptive Streaming over HTTP）。这两类协议均采用视频切片传输机制，即将完整视频划分为多个连续的视频分片，并为每个分片预先生成多个不同码率版本。客户端根据当前网络状态和缓冲区情况，动态选择合适码率的视频分片进行下载，从而在视频质量和播放流畅性之间取得平衡。本文研究的自适应码率问题正是在 MPEG-DASH 类视频传输框架下进行建模与优化的。

在自适应码率视频传输过程中，客户端并非一次性获取完整视频，而是按照播放进度逐段下载视频分片。每次下载新分片前，客户端会根据已观测的网络吞吐量、当前缓冲区长度、剩余视频长度以及不同码率下的分片大小等信息，选择下一分片对应的码率版本。

在带宽较充足时，客户端倾向于选择高码率以提高画质；在带宽下降或缓冲区不足时，则需要降低码率以减少卡顿风险。由于重缓冲对用户体验的影响通常大于短时间画质下降，ABR 算法一般会对卡顿设置较高惩罚。码率频繁切换也会造成画面质量波动，平滑性因此成为 QoE 评价中的重要组成部分。

1.2 ABR 决策问题建模

从优化目标看，ABR 决策的难点不仅在于网络带宽变化，也在于多个 QoE 指标之间存在制约关系。平均码率对应视频画质，重缓冲时间对应播放连续性，平滑性惩罚对应码率稳定性。提高码率可以改善清晰度，但会增加下载时间和卡顿风险；降低码率可以保护缓冲区，却会牺牲画质；频繁切换码率有助于适应带宽变化，也会带来画质波动。因此，ABR 本质上是在动态网络环境下平衡画质、卡顿和稳定性的序列决策问题。本文采用平均奖励、平均码率、重缓冲时间和平滑性惩罚等指标对策略进行综合评价。

在仿真环境中，网络轨迹决定可用吞吐量，视频分片大小决定下载成本，缓冲区状态影响后续卡顿可能性。当策略在短时带宽升高时选择高码率，可能因后续带宽下降和缓冲区不足产生重缓冲；相对保守的策略虽然平均码率较低，但能够维持更安全的缓冲

区水平。本文实验中“平均码率低于 Genet、但平均奖励更高”的现象，说明综合奖励函数并不单纯追求高码率，而是对卡顿风险进行了更强约束。

ABR 可以建模为一个离散时间序列决策问题。在第 t 个视频分片下载前，客户端观测状态 s_t ，并选择动作 a_t 。动作空间通常对应若干个离散码率等级，例如从最低清晰度到最高质量的 6 个码率档位。环境根据所选码率、网络吞吐量和分片大小计算下载时间，并更新缓冲区状态，同时给出奖励 r_t 。

1.3 强化学习与离线策略学习

强化学习适用于 ABR，是因为它能够在学习策略时考虑当前码率选择对后续缓冲区、卡顿和收益的影响。传统规则方法多根据缓冲区阈值或短期吞吐估计做出当前决策，容易忽略连续多个分片之间的长期影响；强化学习方法则可以通过累计奖励比较不同动作序列的效果，从而学习更适合长期 QoE 的码率选择策略。不过，强化学习模型的表现仍会受到训练轨迹、奖励函数和模型结构的影响。训练数据如果缺少弱网或突发波动场景，模型在测试时可能出现性能不稳定。本文选取 Genet 作为重要基线，主要是因为其通过自动课程生成扩展了训练环境覆盖范围，能够代表性能较强的学习型 ABR 方法。

本文的大模型策略采用离线训练方式，不在真实网络环境中进行在线探索。在线探索可能尝试过高码率并引发卡顿，从而影响用户体验；离线经验池能够固定训练数据来源，使实验过程更容易复现。经验池包含状态、动作和奖励等序列信息，模型通过学习状态上下文与码率动作之间的关系完成策略训练。离线训练效果仍取决于经验池质量，本文仅在当前实验协议下讨论 QoE 提升效果，不将结果直接推广到所有真实网络场景。

1.4 Transformer 与大语言模型

Transformer 以自注意力机制为核心^[20]，能够在序列内部建立长距离依赖关系。对于输入序列中的查询（Query）、键（Key）和值（Value），其注意力计算过程可表示为：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (1.1)$$

其中， Q 、 K 和 V 分别表示查询矩阵、键矩阵和值矩阵； d_k 表示键向量的维度； $\text{Softmax}(\cdot)$ 用于将注意力得分归一化为概率分布。该机制能够根据输入序列中不同位置之间的相关性，自适应地分配信息权重，从而有效捕获长距离依赖关系。大语言模型通常在大规模文本语料上进行预训练，因此其输入空间和表征空间具有明显的语言语义特征。将大语言模型用于 ABR 任务时，直接输入原始数值状态会造成模态差异：模型预训练阶段学

习到的是离散词元之间的语义关系，而 ABR 状态则是连续数值、业务指标和历史序列的组合。

语义重编程是将非文本数值状态转换为大语言模型更容易处理的表示形式。在本文 ABR 任务中，吞吐量历史、缓冲区长度和剩余分片数量不仅是数值特征，也包含具体决策含义。例如，低吞吐量意味着下载负担较重，缓冲区充足表示卡顿风险较低。本文通过语义重编程将这些任务含义融入状态表示，使模型在码率决策时能够同时利用数值信息和语义信息。

除语言模型适配外，时间序列多尺度建模也与本文方法设计密切相关。真实网络吞吐量往往同时包含相邻分片之间的快速波动和一段时间内的趋势变化，单一时间尺度难以完整描述这两类信息。TimeMixer 从细粒度与粗粒度采样尺度出发，对时间序列进行可分解的多尺度混合，为同时刻画局部波动和宏观趋势提供了参考^[13]。本文并不直接使用 TimeMixer 的预测结构，而是借鉴其多尺度分析思想，在 ABR 状态编码中对历史吞吐量进行轻量化多尺度建模。

1.5 参数高效微调与 LoRA

完整微调大语言模型通常需要较高的显存和计算资源，在本科毕业设计的实验条件下难以支持多轮调参。参数高效微调（Parameter-Efficient Fine-Tuning, PEFT）是一类降低大模型训练成本的方法，其基本思想是冻结大部分预训练参数，只训练少量新增参数或任务相关参数，使模型能够以较低成本适配下游任务。与完整微调相比，参数高效微调可以减少可训练参数规模，降低显存占用，并提高多轮实验的可行性。

LoRA^[21]是一种常用的参数高效微调方法。它不直接更新完整的预训练权重，而是在原始权重旁引入低秩增量，用少量可训练参数表示模型在下游任务中的更新方向。设原始权重矩阵为 W ，LoRA 更新后的权重可表示为：

$$W' = W + \Delta W \quad (1.2)$$

$$\Delta W = \frac{\alpha}{r} BA \quad (1.3)$$

其中， ΔW 表示任务适配时引入的低秩增量， A 和 B 为可训练低秩矩阵， r 为低秩分解的秩， α 为缩放系数。由于训练时只更新 A 和 B 等少量参数，而不直接全量更新 W ，因此能够降低显存占用和训练成本。同时，考虑到计算资源的限制，本文选取参数规模较小的 Llama-3.2-1B 作为基座模型，并采用参数高效微调方式训练任务相关参数。该模型参数规模低于 7B，能够满足本课题对模型规模的约束，同时兼顾模型表征能力、实验可运行性和工程复现成本。

2 需求分析与总体设计

2.1 系统目标与设计原则

系统目标是构建一个面向 ABR 任务的大语言模型适配实验平台，用于完成数据输入、基线评测、经验池构造、模型训练、在线测试和结果统计。结合本课题的研究内容，系统需要支持多类 ABR 状态特征输入，并完成大语言模型适配、策略训练和性能验证。

系统设计主要考虑四点。统一性要求传统方法、强化学习方法和大模型方法在相同测试协议下进行评估；可复现性要求训练脚本、测试脚本和日志输出尽量固定参数与随机种子；模块化要求状态编码、策略模型、数据集构造和评测统计相互独立，便于后续替换和消融；任务相关性要求模型改进围绕 ABR 的时序特征、多源状态和 QoE 优化目标展开，避免脱离具体任务进行泛化设计。

2.2 系统总体架构

本文系统由五个层次组成：环境层、基线层、数据层、大模型适配层和评测层。环境层负责根据网络轨迹和视频分片信息模拟下载、播放和缓冲区变化；基线层集成基于缓冲区的 BBA、模型预测控制方法 MPC，以及 UDR、Genet 等学习型 ABR 策略；数据层根据交互轨迹构造离线训练样本；大模型适配层包含状态编码器、语言模型、LoRA 适配参数和策略输出头；评测层统计平均奖励、平均码率、重缓冲时间和平滑性惩罚等指标。各对比方法的具体介绍将在第 5.2 节中说明，系统架构如图 2.1 所示。

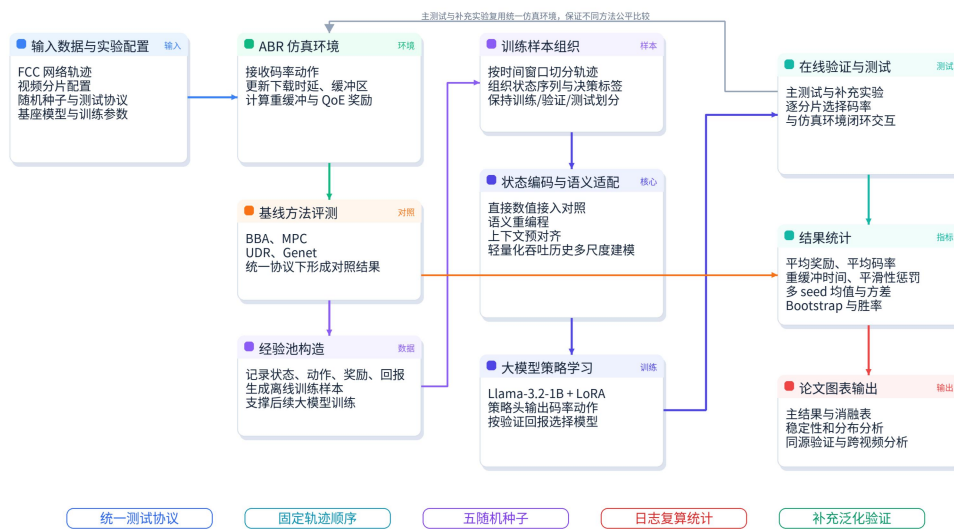


图 2.1 系统架构图

2.3 实验流程设计

实验流程主要包括基线测试、经验池生成、模型训练、验证和测试五个阶段。首先，在统一的仿真环境中对 BBA、MPC、UDR 和 Genet 等对比方法进行测试。所有方法使用相同的网络轨迹和视频分片大小文件，并按照相同格式记录码率、缓冲区、重缓冲时间、下载时延、平滑度和 QoE 奖励等结果，从而保证不同方法之间可以公平比较。

在仿真过程中，网络轨迹提供不同时刻的可用吞吐量，视频元数据提供每个码率档位下的视频分片大小。环境会根据当前选择的码率计算该视频分片的下载时间，并更新缓冲区状态。如果下载时间超过当前缓冲区长度，就会产生重缓冲；如果缓冲区过大，环境会模拟等待过程。每下载完一个分片后，环境会生成新的状态信息，包括上一分片码率、当前缓冲区、历史吞吐量、历史下载时间、下一分片在不同码率下的大小以及剩余分片数量。

经验池由已有基线策略或控制器与 ABR 环境交互生成。每一步交互记录决策前状态、最终选择的码率档位、该动作对应的即时奖励以及视频结束标记。其中，动作统一编码为最终码率类别，取值范围为 0 到 5。考虑到视频首个分片尚未形成完整的历史上下文，经验池构建阶段会在每条轨迹结束后去除首个分片对应的样本；这一处理与评测阶段跳过首个奖励的统计方式保持一致，从而保证训练样本与测试指标具有相同口径。

在训练阶段，数据集模块首先根据视频结束标记识别轨迹边界，并由奖励序列计算折扣累计回报和时间步索引。随后，程序在展平后的经验序列上构造固定长度上下文窗口，输入状态编码器和大语言模型进行学习；因此，训练样本并不是直接以完整播放轨迹为单位输入模型。状态编码器负责将原始状态中的六类数值信息转化为模型可处理的特征表示，大语言模型结合状态上下文、历史动作、历史奖励和目标回报条件，学习当前状态下不同码率动作的选择规律。

验证阶段在固定验证环境中进行。模型根据当前状态连续做出码率选择，环境返回对应的 QoE 奖励。程序会累计验证过程中的奖励，并保存验证回报最高的模型。这里的 `best_return` 表示训练过程中模型在验证环境上取得过的最高累计回报。测试阶段加载该最优模型，在测试轨迹上进行在线决策，并输出平均 QoE、码率、重缓冲和平滑度等指标。

2.4 评价指标设计

评价指标应围绕研究问题设置，而不是简单列出环境输出。平均奖励用于判断综合 QoE 是否提升，平均码率反映画质收益，重缓冲时间衡量播放连续性，平滑性惩罚衡量码率切换稳定性。四个指标需要结合分析：平均奖励较高但重缓冲也较高时，提升可能

主要来自码率项；重缓冲较低但平均码率过低时，策略可能过于保守；平滑性惩罚过高时，即使码率较高，也会影响观看稳定性。本文在第 5 章结合主结果表、QoE 组成图和逐轨迹分析，避免单一均值掩盖策略行为差异。

本文的综合奖励由视频质量收益、重缓冲惩罚和平滑性惩罚共同组成，与测试奖励计算保持一致。第 t 个分片的即时奖励定义为：

$$r_t = \frac{q(a_t)}{1000} - \lambda R_t^{\text{rebuf}} - \mu \frac{|q(a_t) - q(a_{t-1})|}{1000} \quad (2.1)$$

其中， $q(a_t)$ 表示动作 a_t 对应的视频码率，单位为 Kbps； R_t^{rebuf} 表示第 t 个分片下载过程中产生的重缓冲时间，单位为秒； $\lambda = 4.3$ 为重缓冲惩罚系数； $\mu = 1$ 为平滑性惩罚系数； $|q(a_t) - q(a_{t-1})|$ 表示相邻两个分片之间的码率变化幅度。该公式体现了 ABR 任务中“提高清晰度、减少卡顿、避免频繁切换码率”的综合优化目标。

为整体评价模型在测试集上的表现，本文进一步统计所有测试轨迹上的平均奖励。平均奖励计算公式为：

$$\bar{r} = \frac{1}{N(T-1)} \sum_{n=1}^N \sum_{t=2}^T r_t^{(n)} \quad (2.2)$$

其中， $\bar{r}$ 表示测试集平均奖励， N 表示测试 trace 数量， T 表示每条视频的分片数量， $r_t^{(n)}$ 表示第 n 条 trace 上第 t 个分片的即时奖励。求和从 $t = 2$ 开始，是因为实验统计时跳过了每条 trace 的第一步奖励，以减小初始默认码率 a_0 对平均结果的影响。

上述评价指标从 QoE 收益、播放连续性和码率稳定性等多个角度刻画 ABR 策略性能。其中，平均奖励用于反映整体 QoE 水平，其余指标用于分析奖励变化的具体来源。后续实验将基于这些指标对不同方法进行对比，并结合奖励组成和轨迹表现进一步分析模型决策行为。

3 基于大语言模型适配的 ABR 决策方法设计

3.1 基础策略模型设计

基础策略模型采用“回报—状态—动作”的序列组织方式，将 ABR 决策过程转化为条件序列建模问题。模型根据历史回报条件、历史状态以及历史动作信息，学习在当前网络状态与视频状态下选择下一分片码率。相比在线交互式强化学习训练，该方式更适合利用离线经验池对大语言模型进行任务适配，也便于在输入序列中显式融入回报目标，从而引导模型向高质量决策轨迹学习。

在基础策略模型中，模型根据当前 ABR 状态、历史动作以及回报条件，对当前分片的码率选择进行建模，并输出离散动作 a_t 。为更清晰地描述模型输入与码率决策之间的关系，本文将第 t 个分片的决策过程表示为条件策略分布。设 s_t 为当前 ABR 状态， $R_{\leq t}$ 为截至当前时刻的回报条件序列， $a_{< t}$ 表示历史动作，则策略模型输出可表示为：

$$\pi_{\theta}(a_t | R_{\leq t}, s_{\leq t}, a_{< t}) = \text{Softmax}(W_p h_t + b_p) \quad (3.1)$$

其中， $\pi_{\theta}(\cdot)$ 表示参数为 θ 的码率选择策略， $a_t \in \{0, 1, \dots, K-1\}$ 表示第 t 个分片的离散码率动作， $K=6$ 为码率档位数； h_t 表示大语言模型在当前状态 token 位置输出的隐藏表示； W_p 和 b_p 为策略输出头的可训练参数。该公式对应模型先根据 $R_{\leq t}$ 、 $s_{\leq t}$ 、 $a_{< t}$ 得到隐藏状态 h_t ，再通过线性策略头输出各码率动作概率的过程。

策略头使用交叉熵损失学习专家轨迹或高回报轨迹中的动作选择，同时通过验证集累计回报选择最优模型参数。为便于说明基础策略模型中状态输入、任务适配模块与码率输出之间的关系，本文给出整体结构示意，如图 3.1 所示。

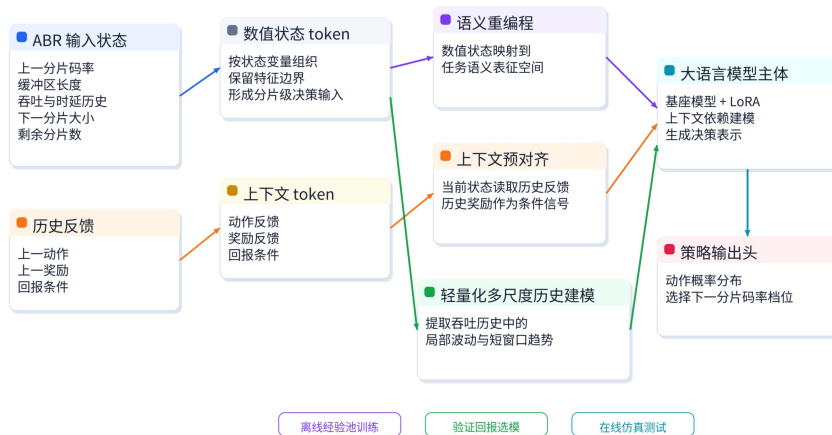


图 3.1 基于大语言模型适配的 ABR 决策总体流程

3.2 语义重编程状态编码

ABR 决策状态通常包括上一分片码率、当前缓冲区长度、历史吞吐量、历史下载时延、下一分片大小和剩余分片数等信息。这些信息虽然都以数值形式输入模型，但在决策中的作用并不相同。上一分片码率主要影响码率切换和平滑性，缓冲区长度反映卡顿风险，历史吞吐量和下载时延反映网络传输能力，下一分片大小决定不同码率下的下载成本。若直接采用统一的数值映射方式，模型可能难以区分不同状态变量的作用，进而影响对复杂网络环境的建模效果。

针对这一问题，本文设计了语义重编程状态编码方法，对 ABR 状态进行语义化表示。该方法不直接把原始数值输入大语言模型，而是根据不同状态变量在 ABR 决策中的含义构建状态表示，再映射到大语言模型的特征空间。由于 ABR 状态由多类含义不同的数值特征组成，本文首先将原始状态矩阵 $s_t \in \mathbb{R}^{6 \times 6}$ 拆分为六类任务特征，并分别进行数值编码：

$$u_{t,i} = f_i(x_{t,i}), \quad i = 1, 2, \dots, 6 \quad (3.2)$$

$$z_{t,i} = \text{LN}(W_i u_{t,i} + p_i) \quad (3.3)$$

其中， $x_{t,i}$ 表示第 t 个分片对应的第 i 类 ABR 特征，六类特征分别为上一分片码率、缓冲区大小、吞吐历史、下载时延历史、下一分片大小和剩余分片数； $f_i(\cdot)$ 表示对应特征的数值编码函数，对于标量特征采用线性映射，对于历史吞吐量和下载时延序列采用卷积编码； $u_{t,i}$ 为数值编码后的特征表示； p_i 为第 i 类特征对应的语义描述嵌入； W_i 为维度映射矩阵。通过引入语义描述嵌入，不同状态变量能够保留其在 ABR 决策中的功能属性，为后续语义对齐提供基础。

与普通线性投影相比，语义重编程更加重视状态变量在任务中的具体作用。普通数值映射主要学习数值到隐藏向量的关系，语义重编程则进一步保留状态变量在 ABR 场景中的语义角色。在得到带有特征语义的信息表示后，本文进一步引入文本语义锚点，使 ABR 状态能够向大语言模型的预训练语义空间对齐。设语义锚点矩阵为 C ，则重编程注意力计算过程为：

$$\alpha_{t,i} = \text{Softmax} \left(\frac{(z_{t,i} W_O)(C W_K)^T}{\sqrt{d_h}} \right) \quad (3.4)$$

$$\tilde{z}_{t,i} = \text{LN}(z_{t,i} + W_O \alpha_{t,i} C W_V) \quad (3.5)$$

其中, $C=[c_1, c_2, \dots, c_M]$ 表示由任务相关文本短语构成的语义锚点集合; W_Q 、 W_K 、 W_V 和 W_O 分别为查询、键、值及输出映射矩阵; $\alpha_{t,i}$ 表示状态特征与语义锚点之间的关联权重; $\tilde{z}_{t,i}$ 表示语义重编程后的状态表示。通过注意力机制, 模型能够根据当前状态动态选择相关语义信息, 从而增强数值状态与语言模型语义空间之间的对应关系。例如, 下载时间增加可能来自带宽下降, 也可能来自视频分片变大, 两种情况都会影响下载时延, 但对应的影响因素并不相同。保留这些语义差异后, 模型可以更准确地判断当前状态与后续码率选择之间的关系。

语义重编程并不依赖自然语言生成, 而是在嵌入空间中建立数值状态与任务语义之间的对应关系, 使大语言模型能够利用上下文建模能力处理 ABR 时序状态。该设计降低了纯数值输入与语言模型预训练空间之间的差异, 也为后续历史上下文建模和多尺度趋势分析提供了更稳定的状态表示基础, 具体过程如图 3.2 所示。

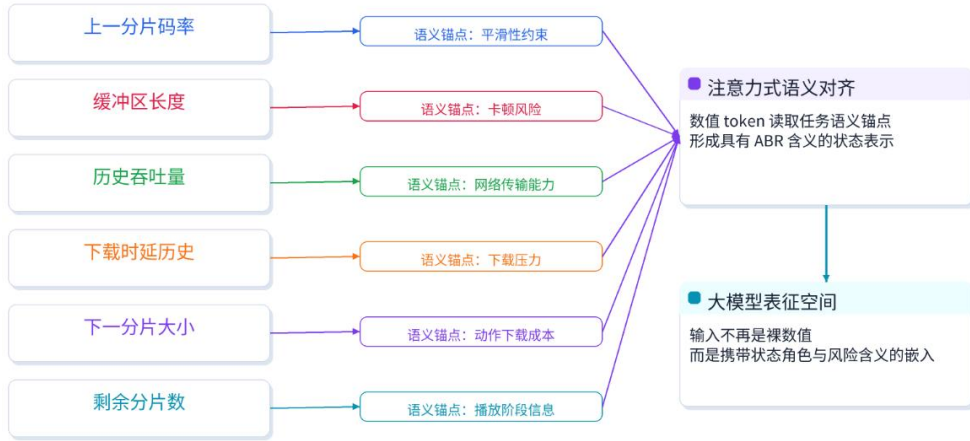


图 3.2 ABR 数值状态的语义重编程映射

3.3 上下文预对齐机制

ABR 决策不仅取决于当前网络状态, 也会受到上一时刻决策结果的影响。上一时刻选择的码率会改变当前缓冲区、重缓冲风险和用户 QoE, 上一动作得到的奖励也能反映该选择在当时网络条件下是否合适。若模型只使用常规状态表示, 而不显式利用上一步反馈, 就难以建立“历史动作—环境反馈—当前决策”的连续关系, 也不容易准确判断当前状态对后续卡顿和码率选择的影响。

针对这一问题, 本文引入上下文预对齐机制, 在进入大语言模型主体前先建立当前状态与历史反馈之间的局部关系。该机制引入上一时刻动作和奖励两类信息, 作为当前

决策的历史条件。历史动作表示上一时刻选择的码率，历史奖励表示该选择带来的 QoE 效果。模型可以同时利用“上一步选择了什么”和“上一步效果如何”来理解当前决策环境。上一较高码率导致奖励下降时，模型更容易转向保守码率；上一较高码率仍获得较好奖励时，当前网络通常更可能支持较高码率。

为了使模型在当前决策前显式感知上一时刻的动作及其结果，本文将上一时刻动作 a_{t-1} 与奖励 r_{t-1} 映射为上下文 token，并在语义对齐前与当前状态 token 进行交互。上下文 token 的构造方式如下：

$$c_t = [W_r r_{t-1} + e_r; W_a a_{t-1} + e_a] \quad (3.6)$$

其中， W_r 和 W_a 为线性映射矩阵， e_r 和 e_a 为上下文类型嵌入， c_t 表示由历史奖励和历史动作构成的上下文表示。

在获得上下文表示后，模型利用注意力机制建立历史反馈与当前状态之间的关联关系，其过程表示为：

$$\hat{U}_t = U_t + \text{Attn}(U_t, c_t, c_t) \quad (3.7)$$

其中， $U_t = [u_{t,1}, u_{t,2}, \dots, u_{t,6}]$ 表示当前时刻的状态 token 集合； $\text{Attn}(Q, K, V)$ 表示注意力计算； $\hat{U}_t$ 表示融合历史反馈后的状态表示。通过该过程，当前状态能够主动读取历史动作和奖励所包含的信息，从而建立状态与历史决策结果之间的联系。

历史反馈并不是简单拼接到输入序列中，而是在预对齐阶段参与当前状态表征。这样处理的重点不是增加输入长度，而是利用历史反馈解释当前状态。相同的缓冲区长度在不同上一奖励条件下可能代表不同卡顿可能性，提前建立这种条件关系有助于模型区分状态背后的决策含义。

由于该机制仅使用上一时刻动作 a_{t-1} 和奖励 r_{t-1} ，不包含当前或未来时刻的信息，因此满足在线 ABR 决策中的因果约束。同时，本文采用受限信息融合方式，使历史反馈作为只读条件信号参与当前状态理解，而不反向更新历史反馈本身，从而减少上下文之间的相互干扰。最终实现中，当前状态主要读取上一时刻奖励反馈，使历史奖励成为判断当前状态是否容易导致卡顿的重要条件信号。

历史动作和历史奖励能够补充当前状态信息，但仍不足以稳定刻画动态网络变化，后续还需要结合吞吐历史趋势，提高策略在复杂网络环境中的稳定性和适应能力。

3.4 轻量化历史多尺度建模

前两种改进主要解决状态语义理解和历史决策反馈利用问题，但对于网络吞吐量序列中的长期趋势信息挖掘仍然有限。ABR 决策高度依赖带宽变化趋势，仅依靠当前状

态和单步反馈难以准确反映网络演化过程，因此进一步设计轻量化历史多尺度建模模块。

吞吐量历史是 ABR 决策中的重要时间序列特征。实际网络带宽通常存在波动，这种波动可能是短时间抖动，也可能是持续一段时间的上升或下降趋势。短时抖动不一定需要立即降低码率，持续下降则可能意味着后续缓冲区压力增加。只使用单一尺度的历史表示，难以同时反映局部变化和整体趋势。

针对这一问题，本文设计轻量化历史多尺度建模模块，对历史吞吐量序列进行多尺度特征提取。这里的“尺度”主要指观察吞吐量变化的历史时间窗口长度。较短时间窗口更关注相邻分片之间的快速波动，较长时间窗口更关注一段时间内的整体变化趋势。对于 ABR 决策而言，短时波动不一定需要立即降低码率，而持续下降趋势可能意味着后续缓冲区更容易被消耗。通过融合不同时间窗口下的吞吐量信息，模型能够同时利用局部变化和短窗口趋势，从而更完整地表示网络状态变化过程。

为了在较低计算开销下同时刻画吞吐量的趋势信息与波动信息，本文对最近 $L_h = 6$ 个分片的吞吐历史序列进行多尺度下采样，并在每个尺度上分别提取趋势项和波动项，其过程表示为：

$$H_t^{(l+1)} = \text{AvgPool}(H_t^{(l)}), \quad l = 1, 2, \dots, L-1 \quad (3.8)$$

$$T_t^{(l)} = \text{MA}(H_t^{(l)}) \quad (3.9)$$

$$S_t^{(l)} = H_t^{(l)} - T_t^{(l)} \quad (3.10)$$

其中， $H_t^{(1)}$ 表示原始吞吐历史序列， $H_t^{(l)}$ 表示第 l 个尺度下的吞吐序列； $\text{AvgPool}(\cdot)$ 表示平均池化下采样； $\text{MA}(\cdot)$ 表示移动平均操作； $T_t^{(l)}$ 表示趋势项； $S_t^{(l)}$ 表示波动项。通过同时保留趋势信息和波动信息，模型能够区分持续性带宽变化与短时网络抖动，从而获得更全面的历史网络状态描述。

在轻量化模块形成前，本文曾尝试原始多尺度历史混合模型。该模型同样面向吞吐量历史序列进行多尺度建模，但结构更完整，包含更多跨尺度混合分支和可训练参数。该版本用于消融实验中的早期对照，以分析直接引入较复杂多尺度结构的效果和局限，并为后续轻量化设计提供依据。

考虑到 ABR 任务对实时性和推理效率有较高要求，本文没有采用复杂的大规模时间序列预测结构，而是只对吞吐量历史通道进行轻量化增强。不同尺度下的趋势表示与波动表示首先映射到统一特征空间，然后融合为多尺度历史增强表示：

$$m_t = \text{LN}\left(\text{Fuse}\left([g_l^S(S_t^{(l)}) + g_l^T(T_t^{(l)})]_{l=1}^L\right)\right) \quad (3.11)$$

其中, $g_l^s(\cdot)$ 和 $g_l^t(\cdot)$ 分别表示波动项和趋势项的投影函数; $\text{Fuse}(\cdot)$ 表示多尺度特征融合操作; m_t 表示融合后的历史增强表示。

随后, 本文采用残差方式将多尺度历史信息注入原始吞吐量表示:

$$u'_{t,\text{thr}} = u_{t,\text{thr}} + \tanh(g) \odot (Pm_t - u_{t,\text{thr}}) \quad (3.12)$$

其中, $u_{t,\text{thr}}$ 表示原始吞吐历史编码; P 为维度投影矩阵; g 为可学习门控参数; $\odot$ 表示逐元素乘法。由于 $\tanh(g)$ 的初始值接近 0, 多尺度增强信息在训练初期以较保守的方式注入, 从而降低新增模块对原有语义表示的扰动。

相比直接替换原始吞吐量表示, 残差融合方式更加稳定。当多尺度趋势信息有助于决策时, 模型能够增强其作用; 当历史趋势贡献有限时, 原始状态表示仍然能够保留主要影响。该模块作为辅助历史表示参与当前状态编码, 并不直接预测未来带宽, 从而降低模型复杂度, 也减少显式带宽预测误差对后续决策的影响。

轻量化历史多尺度建模模块以较低复杂度增强了模型对吞吐历史变化的表示能力, 使策略在当前决策时能够同时参考局部波动和短窗口趋势。相比单纯扩大模型规模, 针对 ABR 场景进行历史趋势增强更符合本文实验规模和任务特点。上下文预对齐机制与轻量化多尺度历史建模模块共同构成本文的历史增强结构, 整体关系如图 3.3 所示。

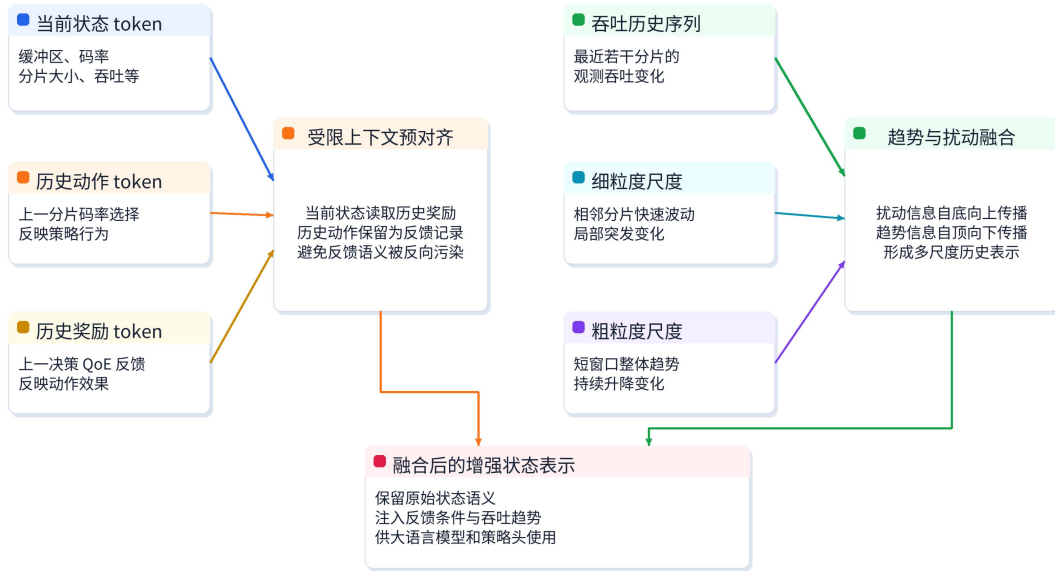


图 3.3 历史反馈与吞吐趋势增强模块

3.5 训练目标与优化方法

训练阶段以离线轨迹中的动作标签为监督信号，使用交叉熵损失优化策略输出。设专家动作或高回报轨迹动作为 a_t^* ，则训练目标为最小化：

$$\mathcal{L}_{\text{CE}}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log \pi_{\theta}(a_t^* | R_{\leq t}, s_{\leq t}, a_{< t}, r_{< t}) \quad (3.13)$$

其中， $\mathcal{L}_{\text{CE}}(\theta)$ 表示交叉熵损失， T 表示训练序列长度， a_t^* 表示经验池中第 t 个时间步的动作标签， $\pi_{\theta}(a_t^* | R_{\leq t}, s_{\leq t}, a_{< t}, r_{< t})$ 表示模型赋予真实动作 a_t^* 的概率。最小化该损失等价于提高模型在相同上下文条件下选择经验轨迹动作的概率。

为降低大语言模型训练过程中的显存占用和计算成本，本文在优化阶段采用第 1.5 节介绍的 LoRA 参数高效微调方法。具体实现中，预训练语言模型的主体参数被冻结，仅在部分注意力映射层中加入可训练的低秩适配参数。本文中的 LoRA 主要作用于注意力模块中的查询映射层和数值映射层，即 `q_proj` 和 `v_proj`。同时，状态编码器、时间步嵌入、回报嵌入、动作嵌入以及最终码率预测层参与训练，使模型能够根据 ABR 状态序列输出码率选择结果。

优化器采用 AdamW，并通过权重衰减抑制过拟合。训练过程中使用学习率预热策略，使学习率在训练初期逐步增大，从而提高训练稳定性。考虑到大语言模型训练时显存占用较高，实验中还采用梯度累积方式等效扩大 batch size，并对梯度范数进行裁剪，减少训练过程中的梯度波动。

训练过程中，模型会定期在验证环境中进行在线决策，并计算该阶段获得的累计验证回报。若当前累计回报高于历史最优值，则保存当前模型参数作为最优模型。这里的模型选择不只依据训练损失大小，而是结合验证环境中的实际决策效果进行判断。这样可以避免仅根据离线拟合效果选择模型，使最终保存的模型更加符合 ABR 任务中提升 QoE 回报的目标。

4 系统实现

4.1 实验环境与工程结构

本文实验系统基于自适应码率视频传输仿真环境构建，并结合大语言模型训练框架完成策略学习和在线测试。系统采用模块化设计，将环境仿真、数据处理、模型训练和结果评估分开实现，以提高实验流程的可复现性和后续扩展性。

系统底层环境负责读取网络轨迹、模拟视频分片下载、更新缓冲区状态并计算 QoE 奖励。模型部分基于 PyTorch 和 Transformer 框架实现，结合参数高效微调方法完成大语言模型适配。整体流程按照“环境交互—经验采样—离线训练—在线验证”展开，使传统方法、强化学习方法和大模型方法能够在统一测试协议下进行比较。

工程实现中，系统主要包括环境模块、数据模块、模型模块和实验脚本模块。环境模块模拟网络变化和状态转移，数据模块组织经验轨迹并构造训练样本，模型模块完成状态编码、上下文建模和策略输出，实验脚本模块负责训练调度、验证评估和测试统计。该结构降低了不同实验之间的耦合程度，也便于后续替换状态编码器或策略结构。

4.2 ABR 环境与基线测试实现

ABR 环境模块用于模拟动态网络下的视频分片下载和播放过程。系统在每个时间步接收当前码率动作，并根据网络轨迹中的可用吞吐量、对应码率下的视频分片大小和链路时延计算分片下载时间。下载完成后，环境根据下载时间更新客户端缓冲区：当下载时间超过当前缓冲区可支撑的播放时长时，超出部分记为重缓冲时间；当下载时间未超过缓冲区长度时，视频继续播放并消耗相应缓冲。每个分片下载完成后，缓冲区增加一个固定分片时长；若缓冲区超过上限，环境会模拟等待过程，以避免缓冲区无限增长。

环境同时维护视频播放进度和下一分片大小等状态信息。每一步交互后，系统更新上一分片码率、缓冲区长度、历史吞吐量、历史下载时间、下一分片在不同码率档位下的大小以及剩余分片数。这些信息共同构成后续策略决策所需的 ABR 状态。即时奖励由码率收益、重缓冲惩罚和平滑性惩罚组成，其中重缓冲惩罚系数为 4.3，平滑性惩罚系数为 1。该奖励形式使策略在提高视频质量的同时，需要控制卡顿时间和码率频繁切换带来的体验损失。

在统一环境下，本文将 BBA、MPC、UDR 和 Genet 等对比方法接入相同的分片级决策接口。每个时间步，环境向策略提供当前 ABR 状态，策略返回码率动作或可转换为码率档位的动作表示；随后，环境根据该动作执行分片下载，并更新缓冲区、播放进度和即时奖励。对于规则类或控制类方法，输出结果可直接对应具体码率档位；对于学

习型方法，已训练神经网络输出的动作结果需要经过动作映射，转换为最终选择的码率档位。通过统一接口封装，不同类型方法能够在相同环境逻辑下完成测试。

ABR 方法的性能差异不仅取决于策略本身，也受到测试协议影响。若不同方法使用不同网络轨迹、视频分片大小或指标计算方式，平均奖励差异可能来自测试条件变化，而不一定来自算法能力。为减少实验偏差，本文将所有方法放在相同测试协议下评估，统一网络轨迹数量、视频配置、随机种子设置和评价指标，并记录逐分片播放日志。后续实验基于这些日志统计平均奖励、平均码率、重缓冲时间和平滑性惩罚，从而保证不同方法之间的结果具有可比性。

4.3 经验池与数据集构造

本文采用离线策略学习方式训练大模型策略，经验池质量会直接影响模型学到的决策行为。本文训练使用项目默认经验池，该经验池由已有基线策略或控制器与 ABR 仿真环境交互生成。交互过程中，环境根据当前码率动作、网络吞吐量和视频分片大小模拟分片下载、缓冲区变化和重缓冲过程，并返回即时奖励和视频是否结束等信息。经验池记录的基本字段包括状态、动作、即时奖励和终止标记。其中，状态包含上一分片码率、缓冲区长度、历史吞吐量、历史下载时间、下一分片大小以及剩余分片数等信息；动作表示最终选择的码率档位，取值范围为 0 到 5；即时奖励由码率收益、重缓冲惩罚和平滑性惩罚共同计算得到；终止标记用于区分不同播放轨迹的边界。

从数据规模看，本文使用的默认经验池共包含 19928 条训练样本，对应 424 条完整播放轨迹。经验池生成阶段会跳过每条播放轨迹的第一个分片样本，因此每条轨迹最终保留 47 个时间步。数据集构造时，系统首先根据终止标记识别不同播放轨迹的边界，并在每条轨迹内部根据奖励序列计算折扣累计回报；随后将展平后的经验序列切分为固定长度上下文窗口，形成适合大模型输入的训练样本。需要说明的是，固定长度窗口基于展平后的经验序列构造，终止标记主要用于区分轨迹边界、计算累计回报和时间步索引；因此，训练样本并不是直接以完整播放轨迹为单位输入模型。这样构造得到的样本不仅包含当前状态与动作标签，还通过回报信息反映后续 QoE 结果，使模型在离线训练过程中能够学习状态、动作选择和长期体验质量之间的关系。

ABR 决策具有明显时序依赖，单一时刻状态难以充分反映近期网络变化和历史决策影响。因此，本文将展平后的经验序列划分为固定长度上下文窗口，使模型在当前状态之外能够利用有限历史信息，包括带宽变化趋势、历史动作及其反馈。窗口长度过短会削弱趋势建模能力，过长则会增加训练开销，并引入与当前决策关联较弱的远期信息。该设计在时序信息利用和计算复杂度之间取得折中。

训练样本保留剩余视频分片数等播放进度信息。视频前段剩余播放时间较长，过早消耗缓冲区可能增加后续卡顿风险；视频后段剩余分片较少，策略可以更充分地利用已有缓冲区。加入播放进度信息后，模型能够区分不同播放阶段下相同网络状态对后续卡顿和码率选择的影响。

样本构造只使用当前及历史可观测信息，不引入未来吞吐量、未来动作或未来奖励，以避免训练阶段使用在线决策时无法获得的信息，保证离线训练与在线测试条件一致。本文主要训练超参数设置如表 4.1 所示。

表 4.1 大模型策略训练主要超参数设置

超参数	取值	说明
状态特征维度	256	状态编码器输出的数值特征维度
单状态历史长度	6	每个状态保留最近 6 个时间步的历史信息
上下文窗口长度 w	20	训练时输入模型的固定长度经验序列
折扣因子 γ	1.0	用于计算折扣累计回报
奖励/回报缩放系数	1000	对奖励和回报进行数值缩放
目标回报缩放系数	1.0	构造在线决策时的目标回报
学习率	$1e-4$	AdamW 优化器学习率
权重衰减	$1e-4$	AdamW 权重衰减系数
预热步数	2000	学习率 warmup 步数
训练轮数	60	模型训练 epoch 数
梯度累积步数	32	用于降低显存占用并稳定训练
LoRA 秩 rank	128	低秩适配矩阵秩
重编程注意力头数	4	语义重编程模块中的注意力头数
Dropout	0.1	重编程与预对齐模块中的 dropout 比例
预对齐注意力头数	8	预对齐模块中的注意力头数
预对齐隐藏层维度	1024	预对齐模块前馈网络隐藏层维度
验证间隔	每 2 个 epoch	定期在验证环境中评估并保存最优模型

4.4 状态编码与策略模型实现

本文策略模型主要由状态编码模块、大语言模型主体和策略输出模块组成。状态编码模块负责将 ABR 数值状态转换为适合语言模型处理的表示形式，大语言模型主体用于建模状态之间的上下文关系，策略输出模块根据隐藏表示生成不同码率档位的选择概率。

在基础实现中，各类状态变量先经过数值映射进入统一隐藏空间。为增强模型对不同状态作用的区分能力，本文进一步引入语义重编程机制，使状态特征在进入语言模型前具有更明确的任务含义。例如，缓冲区状态反映后续播放的连续性，吞吐量历史反映

网络变化，分片大小决定不同码率动作的下载成本。通过区分这些状态角色，模型能够更好地学习状态信息与码率选择之间的关系。

在此基础上，本文加入历史动作、历史奖励和历史吞吐趋势等辅助信息。历史动作提供上一时刻码率选择的上下文，历史奖励反映上一决策产生的 QoE 反馈；在最终预对齐实现中，历史奖励作为更主要的条件信号参与当前状态理解，历史动作则保留在序列上下文中。轻量化多尺度历史模块仅作用于吞吐历史通道，用于增强模型对短期网络变化趋势的感知能力。

模型输出阶段采用离散动作空间设计，策略输出模块先生成各码率档位的未归一化得分，再通过 Softmax 函数将其转换为动作概率分布。测试时，模型在固定随机种子条件下按该概率分布抽样选择码率动作，并与 ABR 仿真环境交互。由于每一步码率选择都会影响后续缓冲区状态、重缓冲风险和未来奖励，模型性能需要在完整仿真环境中进行综合评价，不能只依赖离线动作预测准确率判断。

4.5 训练、验证与测试脚本实现

训练阶段，本文根据验证集在线回报选择模型，而不是直接使用最后一个 epoch 的参数。离线交叉熵损失只能反映模型对经验池动作分布的拟合程度，不能完全代表在线 QoE 表现。实际播放过程还会受到状态转移、缓冲区变化、重缓冲惩罚和码率切换惩罚等因素影响。因此，本文保存验证回报最高的模型权重，并在测试阶段统一加载评估。该方式能够降低模型对训练动作分布的过拟合，但模型选择仍可能受到验证集划分影响。为检验这一影响，后续实验分别在同源验证子集和跨视频配置下进行测试。

测试阶段，所有方法均在相同网络轨迹数量、轨迹顺序和视频配置下运行，以减少测试条件差异对算法比较的影响。主实验采用固定测试子集和主视频配置，补充实验分别使用同源验证子集和另一组固定视频配置，用于检查结果是否依赖特定轨迹划分或视频分片大小分布。每次测试生成逐分片播放日志，记录码率动作、缓冲区、重缓冲时间、分片下载信息和即时奖励；后续图表脚本基于原始日志统一计算平均奖励、平均码率、重缓冲时间和平滑性惩罚。对于学习型方法，本文统计多个随机种子的均值和标准差；对于 BBA、MPC 等确定性方法，则在相同测试条件下给出单次评估结果。

5 实验设计与结果分析

5.1 实验设置

本章实验围绕五个问题展开：第一，本文方法相对于传统 ABR 方法和强化学习方法是否能够取得更高的综合 QoE；第二，语义重编程、上下文预对齐和轻量化历史多尺度建模三个模块是否分别具有可解释的性能贡献；第三，多随机种子和逐轨迹统计是否支持实验结果的稳健性；第四，在同一网络数据来源的不同轨迹子集上，本文方法相对于效果较好的强化学习基线是否仍能保持可观察的平均收益；第五，当网络轨迹保持一致而视频分片大小分布发生变化时，最终模型是否仍能保持稳定性能。上述问题分别对应主实验结果分析、QoE 组成指标分析、消融实验、分布级分析、同源子集验证和跨视频配置补充实验。

实验基于 ABR 仿真环境、公开视频网络轨迹和固定视频配置开展。训练、验证和测试分别依赖不同轨迹集合，以降低训练数据与测试环境重叠带来的偏差。主测试阶段使用固定测试子集中的 100 条网络轨迹，并保持轨迹顺序一致。固定顺序的作用是减少测试采样噪声，使不同方法的性能差异主要来自策略本身，而不是来自轨迹抽样差异。补充实验进一步使用同源验证子集，并更换视频配置，用于分析实验结果是否受到轨迹划分和视频分片大小分布的影响。

评价指标包括平均奖励、平均码率、重缓冲时间和平滑性惩罚。平均奖励用于衡量综合 QoE，计算时不计入每条轨迹的第一步 reward；平均码率、重缓冲时间和平滑性惩罚则根据完整播放日志统计。这样处理主要是因为第一步奖励受初始默认码率影响较大，而组成指标仍应反映完整播放过程。本文在全部图表中保持相同的计算方式。

对于包含随机性的学习型方法，本文采用多随机种子统计均值和标准差。Genet、UDR、语义重编程基线、上下文预对齐增强模型、原始多尺度历史混合模型和最终轻量化多尺度历史增强模型均测试 5 个随机种子；BBA 和 MPC 为确定性方法，因此在相同测试条件下给出单次评估结果。由于不同方法的随机种子数量并不完全一致，本文在相关图表中标注 seed 数，并在分析标准差时避免把不同 seed 数量下的稳定性直接等同比较。

5.2 对比方法与评价指标

本文比较三类方法。第一类是传统方法，包括 BBA 和 MPC。BBA 只根据缓冲区占用选择码率，能够体现规则方法的简洁性和稳定性；MPC 根据近期吞吐量估计未来带宽，并在有限预测窗口中搜索码率组合，是典型的模型预测控制方法^{[2][4]}。第二类是强

化学习基线，包括 UDR 和 Genet。UDR 代表在随机环境分布中训练的策略，Genet 通过自动课程生成提高训练环境覆盖度，是当前仓库中较强的学习型基线^[7]。第三类是本文的大模型适配方法，包括语义重编程基线、上下文预对齐增强模型、原始多尺度历史混合模型和轻量化多尺度历史增强模型。

评价指标方面，本文以平均奖励作为综合 QoE 指标，以平均码率、重缓冲时间和平滑性惩罚作为组成指标。平均码率反映视频清晰度收益，重缓冲时间反映播放卡顿程度，平滑性惩罚反映相邻分片码率切换带来的观看不稳定性。由于三类指标之间存在权衡关系，本文在分析中既比较综合平均奖励，也结合组成指标解释不同方法的决策取舍。

本文在实验分析中按照模型结构统一命名各方法，而不直接使用训练脚本中的实验缩写。具体而言，semantic_only 对应“语义重编程基线模型”，maskprevreward 对应“上下文预对齐增强模型”，hmix_v2lite 对应“轻量化多尺度历史增强模型”。后续结果表格与性能分析均采用上述论文名称，以保证方法表述的一致性和可读性。

5.3 主实验结果分析

为比较不同自适应码率方法在统一测试协议下的性能，本文选取传统规则方法、预测控制方法、强化学习方法以及本文方法进行对比实验，结果如表 5.1 所示。

表 5.1 主要方法在 ABR 测试集上的性能比较

Method	Seeds	Mean reward	Bitrate	Rebuffering	Smoothness
BBA	1	0.7642 ± 0.0000	1183.3 ± 0.0	0.0472 ± 0.0000	0.3974 ± 0.0000
MPC	1	0.8485 ± 0.0000	1212.9 ± 0.0	0.0953 ± 0.0000	0.1375 ± 0.0000
UDR	5	0.7509 ± 0.0347	1096.9 ± 3.4	0.0834 ± 0.0081	0.1684 ± 0.0019
Genet	5	0.8587 ± 0.0243	1104.8 ± 6.3	0.0792 ± 0.0043	0.0885 ± 0.0032
Semantic reprogramming baseline	5	0.8068 ± 0.0804	1030.4 ± 69.7	0.0765 ± 0.0210	0.0764 ± 0.0062
Context pre-alignment enhanced model	5	0.8547 ± 0.1000	1015.8 ± 26.7	0.0627 ± 0.0225	0.0746 ± 0.0101
Lightweight multi-scale history enhanced model	5	0.8967 ± 0.0475	1057.0 ± 33.7	0.0610 ± 0.0114	0.0821 ± 0.0241

由表 5.1 可知，本文最终轻量化多尺度历史增强模型在 5 个随机种子下的平均奖励为 0.8967 ± 0.0475 ，高于 BBA 的 0.7642、MPC 的 0.8485、UDR 的 0.7509 ± 0.0347 和 Genet 的 0.8587 ± 0.0243 。该结果说明，在本实验设置中，最终模型在平均奖励上高于

传统规则方法、预测控制方法和较强强化学习基线，说明大语言模型经过任务适配后能够取得更好的平均 QoE 表现。

为更直观地展示不同方法之间的平均 QoE 奖励差异，本文进一步绘制了主结果柱状图，如图 5.1 所示。

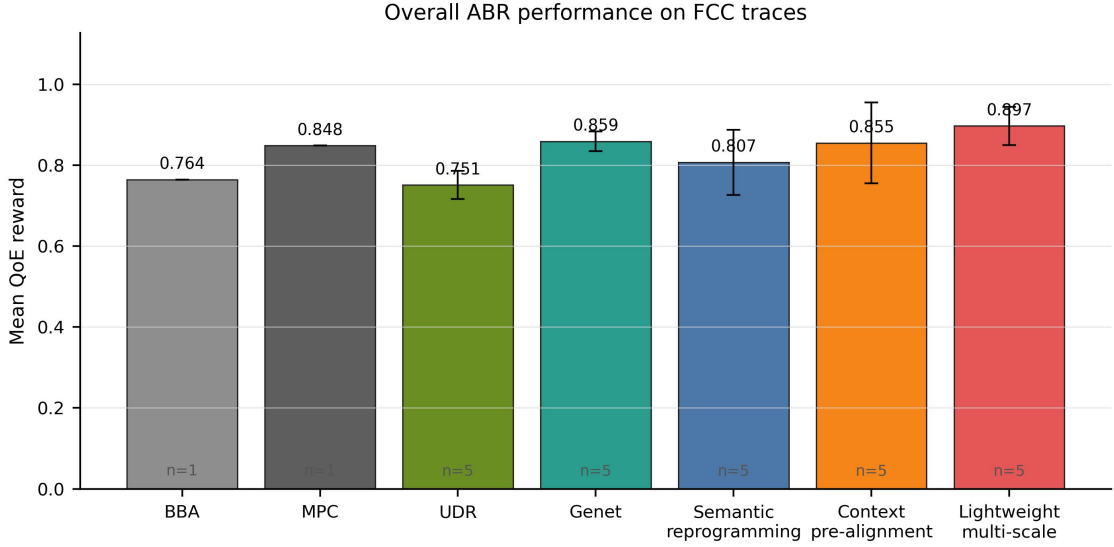


图 5.1 不同 ABR 方法的平均 QoE 奖励对比

由图 5.1 可以看出，各方法的平均奖励存在明显差异。BBA 主要依赖固定缓冲区阈值，平均奖励较低；MPC 利用吞吐量预测进行决策，性能优于 BBA；Genet 在学习型基线中表现较好；本文最终模型取得了更高的平均奖励。最终模型的标准差为 0.0475，高于 Genet 的 0.0243，说明其均值表现更好，但不同随机种子下的结果波动也更明显。这一结果说明，最终模型在 5 个随机种子的均值统计下取得了更高的平均奖励，但由于其标准差高于 Genet，且逐轨迹表现仍存在差异，尚不能说明其在所有随机种子和所有网络轨迹上均稳定优于 Genet。

从方法机理看，最终模型的优势并非来自简单提高码率。其平均码率为 1057.0 Kbps，低于 Genet 的 1104.8 Kbps，却取得了更高的综合奖励。这说明该模型在码率收益、卡顿风险和平滑性惩罚之间形成了更合理的权衡。ABR 任务的目标并不是单纯最大化视频码率，而是在动态网络环境下综合提升用户体验质量；这也解释了为什么 BBA 的平均码率较高，但综合奖励并不占优。

5.4 QoE 组成指标分析

为进一步分析不同方法在 QoE 组成项上的差异，本文比较了平均码率、重缓冲时间和平滑性惩罚三个指标，结果如图 5.2 所示。

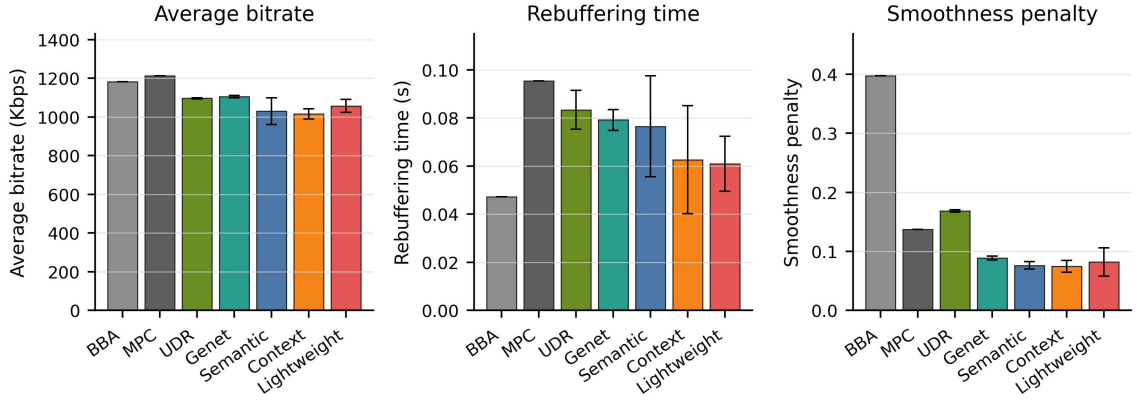


图 5.2 不同方法的平均码率、重缓冲时间和平滑性惩罚对比

由图 5.2 可以看出，不同方法在码率收益、卡顿控制和平滑性保持方面存在明显差异。MPC 的平均码率最高，但其重缓冲时间也相对较高，说明该方法在部分带宽波动较大的轨迹上可能采取了较为激进的码率选择策略。BBA 的平滑性惩罚明显高于其他方法，说明基于缓冲区阈值的规则策略可能导致较频繁的码率切换。相比之下，本文最终模型的平均码率低于 Genet，但重缓冲时间也低于 Genet，最终取得了更高的综合奖励。

这一结果说明，在 ABR 任务中仅分析单一指标是不充分的。如果只关注平均码率，MPC 和 Genet 看起来更具优势；如果只关注重缓冲时间，部分保守策略也可能表现较好。只有将码率收益、卡顿惩罚和平滑性惩罚放在统一的 QoE 框架下分析，才能更准确地解释不同策略之间的性能差异。本文方法的提升主要来自对重缓冲和码率波动的更合理控制，而不是简单地选择更高码率。

从应用角度看，这种取舍更符合实际流媒体系统需求。用户通常对短时间重缓冲较为敏感，过高码率如果引发卡顿，反而可能降低整体体验；但过度保守又会影响视频清晰度。轻量化多尺度历史增强模型通过建模吞吐量历史变化，使策略在网络波动时能够更早感知卡顿风险，在网络相对稳定时保持较高码率，从而提高平均 QoE 奖励。

5.5 消融实验与稳定性分析

为验证不同大模型适配模块对 ABR 决策性能的影响，本文对语义重编程、上下文预对齐以及多尺度历史建模模块进行了消融实验，实验结果如表 5.2 所示。

表 5.2 大模型适配模块消融结果

Method	Seeds	Mean reward	Bitrate	Rebuffering	Smoothness
Semantic reprogramming baseline	5	0.8068 ± 0.0804	1030.4 ± 69.7	0.0765 ± 0.0210	0.0764 ± 0.0062
Context pre-alignment enhanced model	5	0.8547 ± 0.1000	1015.8 ± 26.7	0.0627 ± 0.0225	0.0746 ± 0.0101
Original multi-scale history mixer	5	0.8695 ± 0.0492	1046.6 ± 42.8	0.0636 ± 0.0152	0.0870 ± 0.0260
Lightweight multi-scale history enhanced model	5	0.8967 ± 0.0475	1057.0 ± 33.7	0.0610 ± 0.0114	0.0821 ± 0.0241

由表 5.2 可知，语义重编程基线模型在 5 个随机种子下的平均奖励为 0.8068 ± 0.0804 ，是本文后续模块改进的基础对照。前期实验还测试了直接数值接入版本，该版本仅将原始 ABR 状态通过线性层映射到大语言模型输入空间，在 seed=100003 下的平均奖励为 0.5907，明显低于同 seed 下语义重编程基线模型的 0.9167。该结果说明，将 ABR 数值状态映射到更具任务语义的表示空间，有助于提升大语言模型对码率决策状态的理解能力。上下文预对齐增强模型在 5 个随机种子下的平均奖励为 0.8547 ± 0.1000 ，高于语义重编程基线，但标准差也更大，表明引入历史动作和历史奖励后，模型在部分随机种子下能够获得性能提升，整体稳定性仍有不足。因此，预对齐机制需要结合稳定性约束或更合理的特征融合方式进行评价。

原始多尺度历史混合模型在 5 个随机种子下的平均奖励为 0.8695 ± 0.0492 ，高于语义重编程基线模型和上下文预对齐增强模型，说明多尺度历史信息能够在一定程度上增强模型对网络状态变化趋势的建模能力。结合逐 seed 结果来看，原始多尺度结构具有提升性能上限的潜力，但平均奖励仍低于最终轻量化版本，平滑性惩罚也相对更高。该结果表明，直接引入较完整的多尺度混合结构并不是最优选择，仍需要通过结构收缩降低复杂度并改善特征融合方式。

为更直观地展示各消融模型之间的性能差异，本文进一步绘制了平均奖励对比图，如图 5.3 所示。

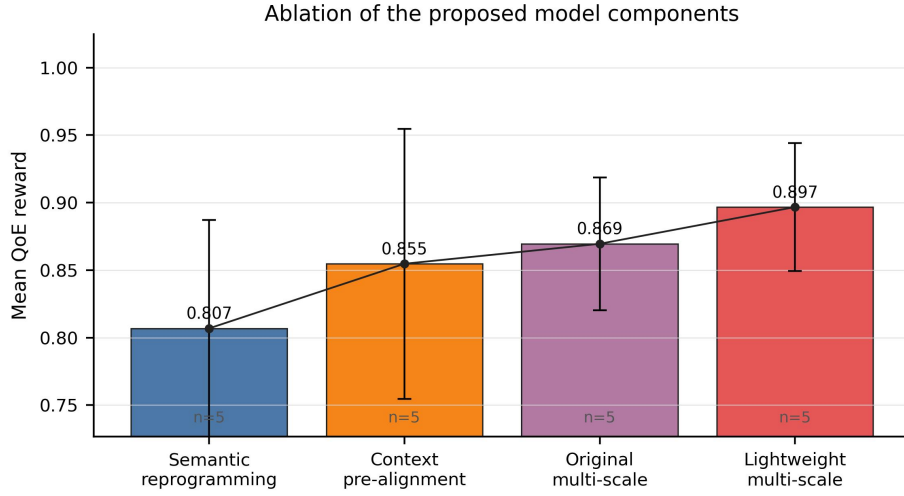


图 5.3 大模型适配模块消融实验

由图 5.3 可以看出，轻量化多尺度历史增强模型的平均奖励达到 0.8967 ± 0.0475 ，是消融实验中平均性能最高的模型。该结果表明，针对吞吐量历史这一关键时间序列特征进行轻量化多尺度建模，并通过门控机制保守注入历史增强特征，能够在保留历史趋势建模能力的同时降低结构复杂度。与原始多尺度历史混合模型相比，该方法在平均奖励和标准差方面均表现更优，说明本文提出的结构收缩策略能够有效提升模型性能和稳定性。

为了进一步分析模型在不同随机种子下的稳定性，本文对 Genet、上下文预对齐增强模型和轻量化多尺度历史增强模型的平均奖励分布进行比较，结果如图 5.4 所示。

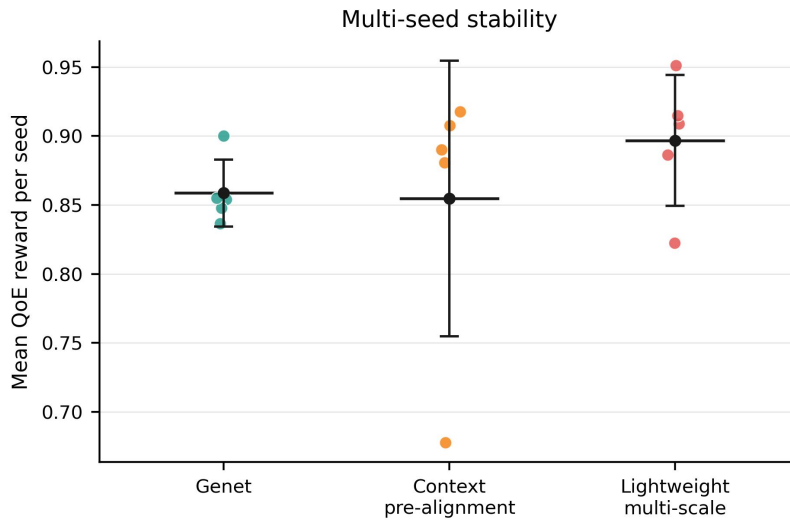


图 5.4 多随机种子下的平均奖励稳定性

由图 5.4 可以看出，Genet 的标准差为 0.0243，说明其在当前测试协议下具有较好的稳定性；上下文预对齐增强模型的标准差为 0.1000，表明该模块单独使用时对随机初始化或训练过程较为敏感；轻量化多尺度历史增强模型的标准差为 0.0475，介于二者之间，说明其在一定程度上缓解了上下文预对齐增强模型带来的性能波动。

上述结果表明，在分析模型性能提升时，需要同时考虑平均性能和稳定性。虽然轻量化多尺度历史增强模型的平均奖励高于 Genet，但其标准差也大于 Genet，说明最终模型在不同随机种子下仍存在一定波动。因此，该结果可以说明最终模型在平均 QoE 上取得提升，但尚不能说明其在所有随机种子和所有测试轨迹上均稳定领先，后续仍需进一步提高模型稳定性。

从方法角度看，轻量化多尺度历史增强模型降低了上下文预对齐增强模型的波动，但尚未完全消除随机性影响。后续工作可从正则化约束、早停策略、随机种子集成以及更稳定的门控初始化等方面进一步降低模型方差，提高不同训练条件下的性能一致性。

5.6 分布级与逐轨迹分析

为了进一步判断平均奖励提升的来源，本文对不同方法在各条测试轨迹上的平均奖励分布进行统计，并绘制逐轨迹平均奖励的累积分布函数（Cumulative Distribution Function, CDF）曲线，结果如图 5.5 所示。

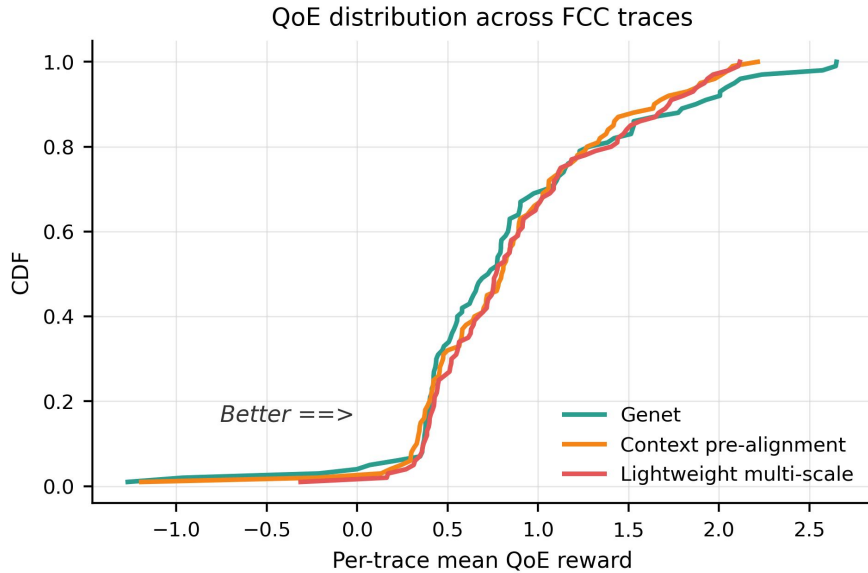


图 5.5 不同方法在测试轨迹上的平均奖励 CDF

由图 5.5 可以看出, CDF 分布图能够反映模型性能提升是来自整体分布右移, 还是主要由少数网络轨迹上的较大收益带来。轻量化多尺度历史增强模型相对于 Genet 在高奖励区间表现更突出, 但在部分轨迹上并未取得优势。这说明本文最终模型的平均奖励提升并不是对所有网络轨迹的全面覆盖, 而是主要来自部分适合历史趋势建模的轨迹上的较大收益。

为进一步量化最终模型相对于 Genet 在逐轨迹层面的差异, 本文统计了两种方法在公共测试轨迹上的奖励差值, 结果如图 5.6 所示。

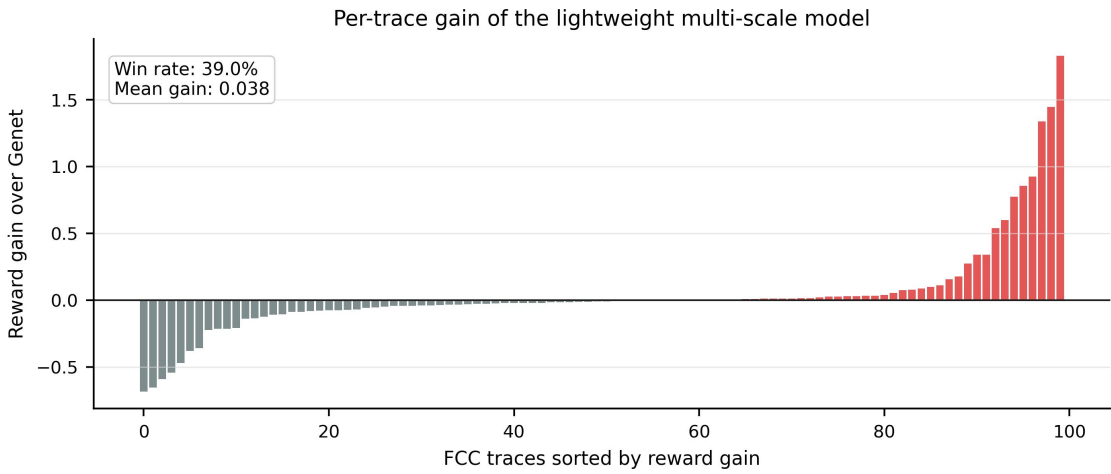


图 5.6 轻量化多尺度历史增强模型相对 Genet 的逐轨迹奖励增益

由图 5.6 可以看出, 最终模型在 100 条公共轨迹中的胜率为 39.0%, 平均奖励增益为 0.0380。虽然最终模型的平均奖励高于 Genet, 但从逐轨迹结果看, 它并不是在大多数网络轨迹上都取得优势。平均值的提升主要来自部分轨迹上较大的正向增益, 而不是所有轨迹上的稳定提升。

从逐轨迹结果看, 轻量化多尺度历史增强模型并未在多数网络轨迹上都取得优势, 但部分轨迹上的正向增益较大, 从而提高了整体平均 QoE。该结果说明, 本文方法的优势主要体现在平均指标上, 逐轨迹层面的稳定性仍有不足, 尚不能说明其在多数网络轨迹下均优于基线方法。后续可从训练稳定性和跨轨迹泛化能力等方面进一步改进。

5.7 同源网络轨迹验证子集补充实验

主实验基于固定测试子集进行评估, 能够比较不同方法在相同测试条件下的平均性能, 但仍可能受到具体轨迹划分的影响。为检验结论是否过度依赖主测试子集, 本文进一步选取与主测试集来源相同、但轨迹划分不同的验证子集进行补充实验。该实验的目

的不是验证跨数据源泛化能力，而是检查方法在同一网络数据来源内部是否仍能保持可观察的平均收益。

实验设置与主实验保持一致：视频配置、评价指标和指标计算方式不变，仅将网络轨迹替换为同源验证子集。本文分别测试 Genet 和最终轻量化多尺度历史增强模型，每种方法均采用 5 个随机种子，并在 100 条网络轨迹上统计平均奖励、平均码率、重缓冲时间和平滑性惩罚。这样的设计能够尽量隔离“轨迹划分变化”这一因素，使结果更适合用于判断主实验结论的稳健性，结果如表 5.3 所示。

表 5.3 FCC 验证集上的补充实验结果

Method	Mean reward	Bitrate	Rebuffering	Smoothness
Genet	0.7355 ± 0.0327	1081.2 ± 4.6	0.1012 ± 0.0069	0.0848 ± 0.0020
Lightweight multi-scale history enhanced model	0.7410 ± 0.0663	1020.3 ± 31.5	0.0868 ± 0.0146	0.0804 ± 0.0243

从表 5.3 可以看出，最终模型在验证子集上的平均奖励为 0.7410，高于 Genet 的 0.7355，绝对提升为 0.0055，相对提升约 0.75%。该结果说明，最终模型在同源验证子集上仍保持小幅均值优势。进一步观察 QoE 组成项可知，最终模型的平均码率从 1081.2 Kbps 降至 1020.3 Kbps，但重缓冲时间从 0.1012 降至 0.0868，约下降 14.2%，平滑性惩罚也略有下降。这表明该子集上的收益主要来自更保守的码率选择和更低卡顿风险，而不是来自更高平均码率。

需要注意的是，该子集上的平均增益明显小于主测试中的 0.0380，说明模型优势对网络轨迹划分具有一定敏感性。该结果对主实验结论起到了一定支撑作用，但不能进一步说明模型已经稳定泛化。结合主测试和同源验证子集结果来看，本文方法在主测试配置下平均 QoE 提升较明显，在同源验证子集上仍有小幅优势；但现有实验还不足以证明其在所有同源轨迹或更广泛网络环境下都能稳定领先。

5.8 跨视频配置补充实验

为进一步分析模型对视频分片大小分布变化的适应能力，本文在保持网络轨迹不变的条件下，将主实验中的视频配置替换为另一组固定视频配置进行补充测试。ABR 决策中的下载时间同时受到网络吞吐量和视频分片大小影响；即使网络轨迹完全相同，不同视频内容也会改变各码率档位对应的下载成本，进而影响缓冲区演化、重缓冲风险和

码率切换代价。因此，该实验主要用于检验最终模型是否对主视频配置存在过度适应，而不是用于验证跨网络来源泛化能力。两组视频配置的分片大小分布如图 5.7 所示。

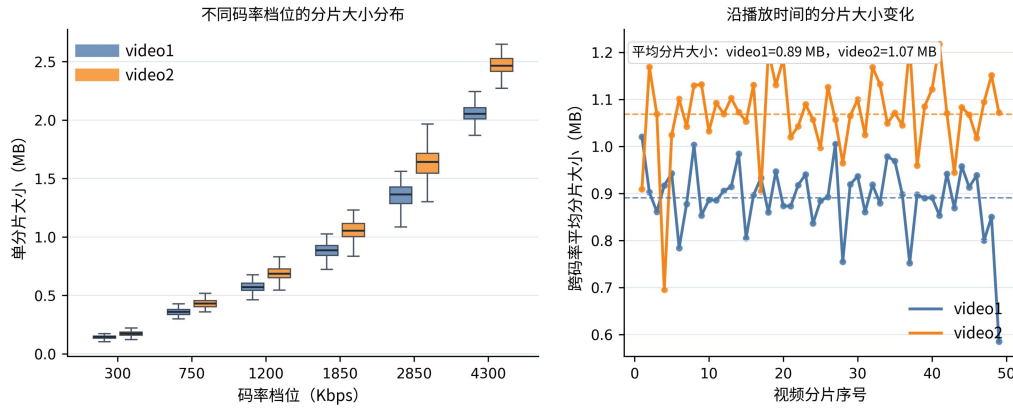


图 5.7 主视频配置与补充视频配置的分片大小分布对比

由图 5.7 可见，补充视频配置在各码率档位下的分片大小整体高于主视频配置，平均分片大小约为主视频配置的 1.20 倍。这意味着在相同网络吞吐条件下，选择相同码率动作时，补充视频配置通常需要更长下载时间，并更容易消耗缓冲区。该差异为后续性能变化提供了直接解释：若模型未能充分感知分片大小增大带来的下载压力，就可能在较高码率选择下产生更多重缓冲。

跨视频配置下的补充实验结果如表 5.4 所示。

表 5.4 跨视频配置下 Genet 与最终模型的补充实验结果

Method	Seeds	Mean reward	Bitrate	Rebuffering	Smoothness
Genet	5	0.6659 ± 0.0274	900.8 ± 5.9	0.0750 ± 0.0065	0.0681 ± 0.0030
Lightweight multi-scale history enhanced model	5	0.6218 ± 0.1510	945.7 ± 26.8	0.0930 ± 0.0368	0.0789 ± 0.0176

从表 5.4 可以看出，在跨视频配置下，最终模型的平均奖励为 0.6218，低于 Genet 的 0.6659，绝对差值为-0.0441。该结果与主视频配置下的正向增益不同，说明本文方法在跨视频配置下尚未形成稳定优势。进一步观察 QoE 组成项可知，最终模型的平均码率高于 Genet，但重缓冲时间和平滑性惩罚也同时升高，说明其在分片大小整体增大的

情况下更倾向于激进码率选择，而较高码率带来的画质收益未能抵消卡顿和平滑性损失。

五个随机种子下的奖励差异如表 5.5 所示。

表 5.5 跨视频配置下五个随机种子的奖励差异

Seed	Genet 平均奖励	最终模型平均奖励	差值	结果解释
100001	0.6617	0.7713	+0.1096	码率略高且重缓冲下降，明显提升
100002	0.7078	0.6192	-0.0886	码率升高导致重缓冲增加，奖励下降
100003	0.6347	0.7174	+0.0827	码率提升同时控制了卡顿，表现较好
100004	0.6736	0.6237	-0.0499	卡顿增加抵消平滑性收益
100005	0.6519	0.3774	-0.2745	策略过激，重缓冲和平滑惩罚均显著增加

表 5.5 进一步给出了不同随机种子下的结果。最终模型并非在所有随机种子上同时退化，其中部分种子仍取得正向增益；但也存在个别种子退化明显，导致整体平均性能下降。这说明最终模型在跨视频配置下仍具有一定性能潜力，但训练稳定性不足，对视频分片大小分布变化较为敏感。换言之，当前模型的问题不是完全无法适应新的固定视频配置，而是在部分初始化条件下容易形成偏激进策略，从而放大重缓冲风险。

典型失败轨迹案例如图 5.8 所示。

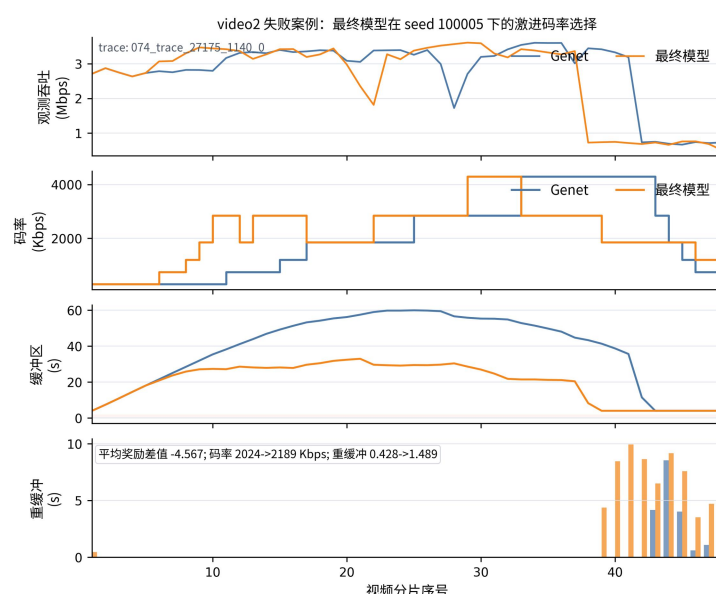


图 5.8 跨视频配置下典型失败轨迹案例分析

图 5.8 展示了退化最明显的案例。在该轨迹中，最终模型在前中段更早提升码率，导致缓冲区余量持续低于 Genet；当后续分片下载压力增大时，低缓冲状态使模型更容易产生集中重缓冲，最终造成平均奖励显著下降。该案例说明，跨视频配置下的性能下降并非来自单一步骤错误，而是由连续偏激进码率选择、缓冲区消耗和后续重缓冲累积共同造成。

从逐轨迹统计看，跨视频配置中最终模型相对 Genet 的胜率为 49.0%，接近一半；但平均增益为-0.0441。这说明模型并不是在大多数轨迹上都失效，而是少数失败轨迹的下降幅度较大，拉低了整体平均结果。结合主视频配置下的结果来看，最终模型在原视频配置下能够提高平均 QoE，但换成分片大小分布不同的视频后，结果不够稳定，暂不能说明其具备可靠的跨视频配置泛化能力。

该实验也为后续改进提供了方向。首先，训练经验池可以引入多视频配置，使模型接触更丰富的分片大小分布，降低对单一视频配置的适应偏差。其次，状态编码中可以加强下一分片大小与可选码率动作之间的显式交互，使模型更容易判断高码率分片带来的下载压力和卡顿可能性。最后，模型选择不宜只依赖单一验证配置下的最高回报，可以考虑加入多视频验证，并将重缓冲时间和结果波动纳入选模依据，以降低个别随机种子下策略过于激进的问题。

5.9 实验局限与方法论反思

为进一步检验实验结论的稳定性，本文使用公共网络轨迹作为配对单位，对最终模型相对 Genet 的平均奖励增益进行 bootstrap 重采样，即自举重采样，结果如表 5.6 所示。

表 5.6 最终模型相对 Genet 的配对 Bootstrap 奖励增益置信区间

实验配置	轨迹数量 n	Genet	最终模型	平均增益	95%CI	P (增益>0)	胜率
FCC-test + video1	100	0.8587	0.8967	0.0380	[-0.0278,0.1106]	85.8%	39%
FCC-valid + video1	100	0.7355	0.7410	0.0055	[-0.0810,0.1052]	52.2%	41%
FCC-test + video2	100	0.6659	0.6218	-0.0441	[-0.1103,0.0219]	9.4%	49%

由表 5.6 可知，在主测试配置下，最终模型的平均增益为 0.0380，但 95%置信区间为[-0.0278, 0.1106]，说明均值提升具有可观察趋势，同时仍存在轨迹级不确定性。在同

源验证子集上，平均增益仅为 0.0055， $P(\text{增益} > 0)$ 为 52.2%，接近统计不确定状态。在跨视频配置下，平均增益为 -0.0441， $P(\text{增益} > 0)$ 为 9.4%，说明当前模型在视频分片大小分布变化时稳定性不足。

基于上述结果，本文实验仍存在以下局限。第一，本文所有网络轨迹均来自同一数据来源，虽然覆盖了主测试子集、同源验证子集和跨视频配置，但尚未覆盖更多网络来源、移动场景、长尾弱网条件和真实播放器部署环境。第二，跨视频配置结果表明，最终模型对视频分片大小分布和随机种子较敏感，说明当前经验池构造和验证选模流程可能仍偏向主视频配置。第三，本文主要关注离线训练和仿真测试，尚未系统评估真实播放器中的推理时延、显存占用、能耗和端侧部署成本。第四，最终模型虽然在主测试配置下取得更高平均奖励，但逐轨迹胜率、同源验证子集的小幅增益和跨视频配置退化共同表明，其稳定性仍不足以支持“全面优于强基线”的结论。

上述局限并不否定本文工作，而是限定了结论边界。本文能够证明的是：在当前仿真环境、数据划分和测试协议下，基于大语言模型适配的轻量化多尺度历史增强模型能够提升主测试配置下的平均 QoE；本文尚不能证明的是：该方法在所有视频内容、所有网络环境和所有部署约束下均优于强化学习基线。将这两类结论区分开，有助于保证实验论证的严谨性。

后续研究可以从三个方向改进。首先，在更多视频配置和网络轨迹来源上补充泛化实验，尤其是跨视频、跨网络来源和长尾弱网场景。其次，针对逐轨迹失败样本分析模型决策失误原因，判断问题来自吞吐历史建模不足、奖励权重设置不充分，还是验证选模目标与测试 QoE 不完全一致。最后，可以结合更轻量的基座模型、模型蒸馏或推理加速方法，降低大模型 ABR 策略的部署成本，使其更接近实际流媒体系统需求。

结 论

本文围绕基于大语言模型适配的自适应码率视频传输决策问题开展研究，完成了 ABR 决策建模、实验环境整理、基线方法评测、经验池构造、大模型策略训练和多随机种子测试分析。本文的核心思路不是直接将数值状态输入大语言模型，而是围绕 ABR 状态的数值性、时序性和反馈性进行任务适配，使模型能够根据网络状态、缓冲区状态和视频分片特征输出合理的码率动作。

方法设计上，本文主要完成了三个方面的工作。第一，针对 ABR 数值状态与大语言模型语义空间不匹配的问题，设计语义重编程状态编码方法，将码率、缓冲区、吞吐历史、下载时延、视频分片大小和剩余视频长度等特征映射到具有任务含义的表征空间。第二，针对历史反馈难以稳定利用的问题，引入历史动作与历史奖励构造上下文预对齐机制，使模型能够在当前决策前感知上一动作及其 QoE 反馈。第三，针对短期带宽趋势建模不足的问题，设计轻量化历史多尺度建模模块，对吞吐历史中的局部波动和趋势变化进行建模，并通过残差门控方式注入状态表示。

实验结果表明，在主测试配置下，轻量化多尺度历史增强模型在 5 个随机种子下取得 0.8967 ± 0.0475 的平均奖励，高于 BBA、MPC、UDR 和 Genet 等基线，其中 Genet 的平均奖励为 0.8587 ± 0.0243 。消融实验说明，语义重编程、上下文预对齐和轻量化多尺度历史建模分别从状态理解、历史反馈利用和吞吐趋势感知三个方面增强了模型能力，共同促进了综合 QoE 提升。

同时，本文实验也揭示了方法边界。主测试逐轨迹分析显示，最终模型相对 Genet 的平均增益为 0.0380，但胜率为 39.0%，说明该方法并非在所有网络轨迹上稳定领先。补充实验进一步表明，最终模型在同源验证子集上仅保持小幅均值优势，在跨视频配置下稳定性不足。因此，本文结论应限定为：在当前仿真环境、数据划分和测试协议下，大语言模型经过任务适配后能够用于 ABR 决策，并在主测试配置中取得更高平均 QoE；但尚不能证明其在所有视频内容、所有网络环境和实际部署约束下均优于强化学习基线。

后续工作可以从三个方向继续改进：一是在更多视频配置和跨网络来源轨迹上验证模型稳健性；二是针对逐轨迹失败样本进行归因分析，判断失败来自吞吐趋势建模不足、目标回报设定偏差，还是状态编码对分片大小和下载压力的刻画不足；三是评估推理延迟、显存占用和参数规模，并结合模型压缩或轻量化推理方法，推动大模型 ABR 策略从仿真验证进一步走向真实播放器系统。

参 考 文 献

- [1] Seufert M, Egger S, Slanina M, et al. A survey on quality of experience of HTTP adaptive streaming[J]. IEEE Communications Surveys & Tutorials, 2015, 17(1): 469-492. DOI: 10.1109/COMST.2014.2360940.
- [2] Huang T Y, Johari R, McKeown N, et al. A buffer-based approach to rate adaptation: Evidence from a large video streaming service[C]//Proceedings of the 2014 ACM Conference on SIGCOMM. New York: ACM, 2014: 187-198. DOI: 10.1145/2619239.2626296.
- [3] Spiteri K, Urgaonkar R, Sitaraman R K. BOLA: Near-optimal bitrate adaptation for online videos[J]. IEEE/ACM Transactions on Networking, 2020, 28(4): 1698-1711. DOI: 10.1109/TNET.2020.2996964.
- [4] Yin X, Jindal A, Sekar V, et al. A control-theoretic approach for dynamic adaptive video streaming over HTTP[C]//Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication. New York: ACM, 2015: 325-338. DOI: 10.1145/2785956.2787486.
- [5] Mao H, Netravali R, Alizadeh M. Neural adaptive video streaming with Pensieve[C]//Proceedings of the 2017 Conference of the ACM Special Interest Group on Data Communication. New York: ACM, 2017: 197-210. DOI: 10.1145/3098822.3098843.
- [6] Huang T, Zhou C, Zhang R X, et al. Comyco: Quality-aware adaptive video streaming via imitation learning[C]//Proceedings of the 27th ACM International Conference on Multimedia. New York: ACM, 2019: 429-437. DOI: 10.1145/3343031.3351014.
- [7] Xia Z, Zhou Y, Yan F Y, et al. Genet: Automatic curriculum generation for learning adaptation in networking[C]//Proceedings of the ACM SIGCOMM 2022 Conference. New York: ACM, 2022: 397-413. DOI: 10.1145/3544216.3544243.
- [8] Zhong Z, Liu M, Yang L, et al. Video streaming with Kairos: An MPC-based ABR with streaming-aware throughput prediction[C]//Proceedings of the 35th Workshop on Network and Operating System Support for Digital Audio and Video. New York: ACM, 2025: 84-90. DOI: 10.1145/3712678.3721886.
- [9] Claeys M, Latré S, Famaey J, et al. Design and evaluation of a self-learning HTTP adaptive video streaming client[J]. IEEE Communications Letters, 2014, 18(4): 716-719. DOI: 10.1109/LCOMM.2014.020414.132649.
- [10] Kim M, Chung K. HTTP adaptive streaming scheme based on reinforcement learning with edge computing assistance[J]. Journal of Network and Computer Applications, 2023, 213: 103604. DOI: 10.1016/j.jnca.2023.103604.
- [11] Wu D, Wang X, Qiao Y, et al. NetLLM: Adapting large language models for networking[C]//Proceedings of the ACM SIGCOMM 2024 Conference. New York: ACM, 2024: 661-678. DOI: 10.1145/3651890.3672268.
- [12] Jin M, Wang S, Ma L, et al. TIME-LLM: Time series forecasting by reprogramming large language models[C]//Proceedings of the International Conference on Learning Representations. 2024.
- [13] Wang S, Wu H, Shi X, et al. TimeMixer: Decomposable multiscale mixing for time series forecasting[C]//Proceedings of the International Conference on Learning Representations. 2024.

- [14] 杜丽娜, 卓力, 杨硕, 等. 基于强化学习的移动视频流业务码率自适应算法研究进展[J]. 通信学报, 2021, 42(9): 205-217. DOI: 10.11959/j.issn.1000-436x.20211178.
- [15] 肖强, 白光伟, 沈航. 强化学习在自适应视频码率控制算法中的应用[J]. 小型微型计算机系统, 2020, 41(2): 252-258.
- [16] 宋靳铎, 张远, 王博. HTTP 自适应流媒体直播系统中的用户体验质量优化[J]. 中兴通讯技术, 2021, 27(1): 48-53. DOI: 10.12142/ZTETJ.202101011.
- [17] 熊丽荣, 雷静之, 金鑫. 基于 Q-learning 的 HTTP 自适应流码率控制方法研究[J]. 通信学报, 2017, 38(9): 18-24. DOI: 10.11959/j.issn.1000-436x.20171178.
- [18] 夏勃, 林浒, 李鸿彬, 等. 一种动态自适应 HTTP 流媒体码率切换算法[J]. 小型微型计算机系统, 2015, 36(2): 275-279.
- [19] Li Y, Zheng Q, Zhang Z, et al. Improving ABR performance for short video streaming using multi-agent reinforcement learning with expert guidance[C]//Proceedings of the 33rd Workshop on Network and Operating System Support for Digital Audio and Video. New York: ACM, 2023: 58-64. DOI: 10.1145/3592473.3592564.
- [20] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems. Red Hook: Curran Associates, 2017, 30: 5998-6008.
- [21] Hu E J, Shen Y, Wallis P, et al. LoRA: Low-rank adaptation of large language models[C]//Proceedings of the International Conference on Learning Representations. 2022.

修改记录

一、毕业论文（设计）题目修改

原题目：基于大语言模型微调的视频自适应码率优化研究

修稿后题目：基于大语言模型微调的视频码率自适应优化

二、毕业论文（设计）内容重要修改记录

1、补充并完善了大语言模型适配 ABR 任务的相关方法说明，重点对语义重编程状态编码、上下文预对齐机制和轻量化历史多尺度建模模块进行了梳理和修改。

2、对实验部分进行了完善，补充多随机种子实验、消融实验、逐轨迹分析、同源验证子集实验和跨视频配置实验，使实验结果能够更全面地反映模型性能和局限。

3、对摘要、英文摘要、图表说明、参考文献格式和正文表述进行了修改，统一 ABR、QoE 等缩写说明，规范图表编号和实验指标表述，减少重复和不够自然的表述。

三、毕业论文（设计）外文翻译修改记录

1、对外文翻译中的专业术语进行了统一和规范，重点统一了大语言模型、时间序列预测、模型重编程、Prompt-as-Prefix、少样本学习和零样本学习等术语的译法，提升译文的专业性和一致性。

2、对 TIME-LLM 框架、patch 重编程、冻结骨干模型、输入变换和输出投影等核心技术段落进行了校对和润色，修正部分直译、生硬表达，使译文内容更加准确，语言更加符合中文论文写作习惯。

四、毕业论文（设计）最后一次检测重复比

最后一次检测重复比为 1%，符合学校毕业论文（设计）检测要求。

记录人（签字）：

指导教师（签字）：

致 谢

行文至此，毕业论文已近尾声，大学四年的时光也即将画上句号。回望来到凌水路的这段求学岁月，有初入大工校园时的懵懂与期待，也有面对课程、实验和论文时的困惑与坚持，但是我收获了我人生中一段十分珍贵的成长经历。一路走来，我曾怀疑过自己，但我总能在一次次的跌倒中重新拾起继续向前的勇气，这离不开老师们的谆谆教诲、同学们的鼎力相助以及家人的默默支持。在此，我谨向所有关心、帮助和鼓励过我的人致以最诚挚的感谢。

首先，衷心感谢我的指导老师和师兄。从论文选题到最终成稿，他们始终给予我耐心而细致的指导。初涉课题时，我对研究内容和论文思路的把握还不够成熟；在后续写作和实验分析过程中，他们多次指出问题，并提出具体可行的建议，帮助我逐渐理清思路、完善论文。通过这次毕业设计，我不仅完成了论文写作，也更加体会到严谨思考、认真求证和踏实做事的重要性。这份教诲与关怀，我将铭记于心。

感谢计算机学院各位老师在本科阶段给予我的培养与教导。四年求学路上，老师们在课堂上传道授业，在实验与实践中的引导我们思考问题、解决问题。正是这些点滴积累，使我逐渐建立起对专业的认识，也让我在面对问题时多了一份耐心、理性与坚持。感谢各位老师一路以来的教诲，使我在不断学习中看见更广阔的方向，也更加坚定了继续前行的信心。

感谢我的同学和朋友们。大学生活因你们的陪伴而更加丰富，也因你们的帮助而少了许多孤单。来自五湖四海的我们，因同一段求学时光相聚于此。最初的陌生，在一次次的相伴、交流与鼓励中慢慢化作熟悉与默契。那些并肩走过的日子，或许没有惊天动地的故事，却以最真切的方式留在了我的青春里，成为大学岁月中温暖而难忘的一笔。

更要感谢我的家人。求学路上，他们始终以最朴素的方式给予我理解、支持与包容。也许他们并不熟悉我的专业内容，却始终牵挂我的学习与生活；也许他们很少说出华丽的鼓励，却用默默守护让我拥有继续前行的勇气。无论今后走向何处，这份来自家人的爱与支持，都将是我最坚实的底气。

四年求学时光倏然而过，论文完成之际，也意味着这段大学岁月即将落下帷幕。回望来路，既有收获的欣喜，也有不足与遗憾；但正是这些经历，让我在一次次思考、尝试和坚持中逐渐成长。山高水长，来日可期。今后无论身处何方，我都将铭记在大工度过的青春岁月，带着这份沉淀与力量，继续以谦逊之心求知，以踏实之态前行。

山水一程，三生有幸。谨以此文，向所有曾经给予我帮助、鼓励与温暖的人表示衷心的感谢。愿此去前路开阔，亦不忘来时初心。